



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Self-Supervised Multimodal Embedding

INFS 4205/7205 Advanced Techniques for High Dimensional Data

Yadan Luo

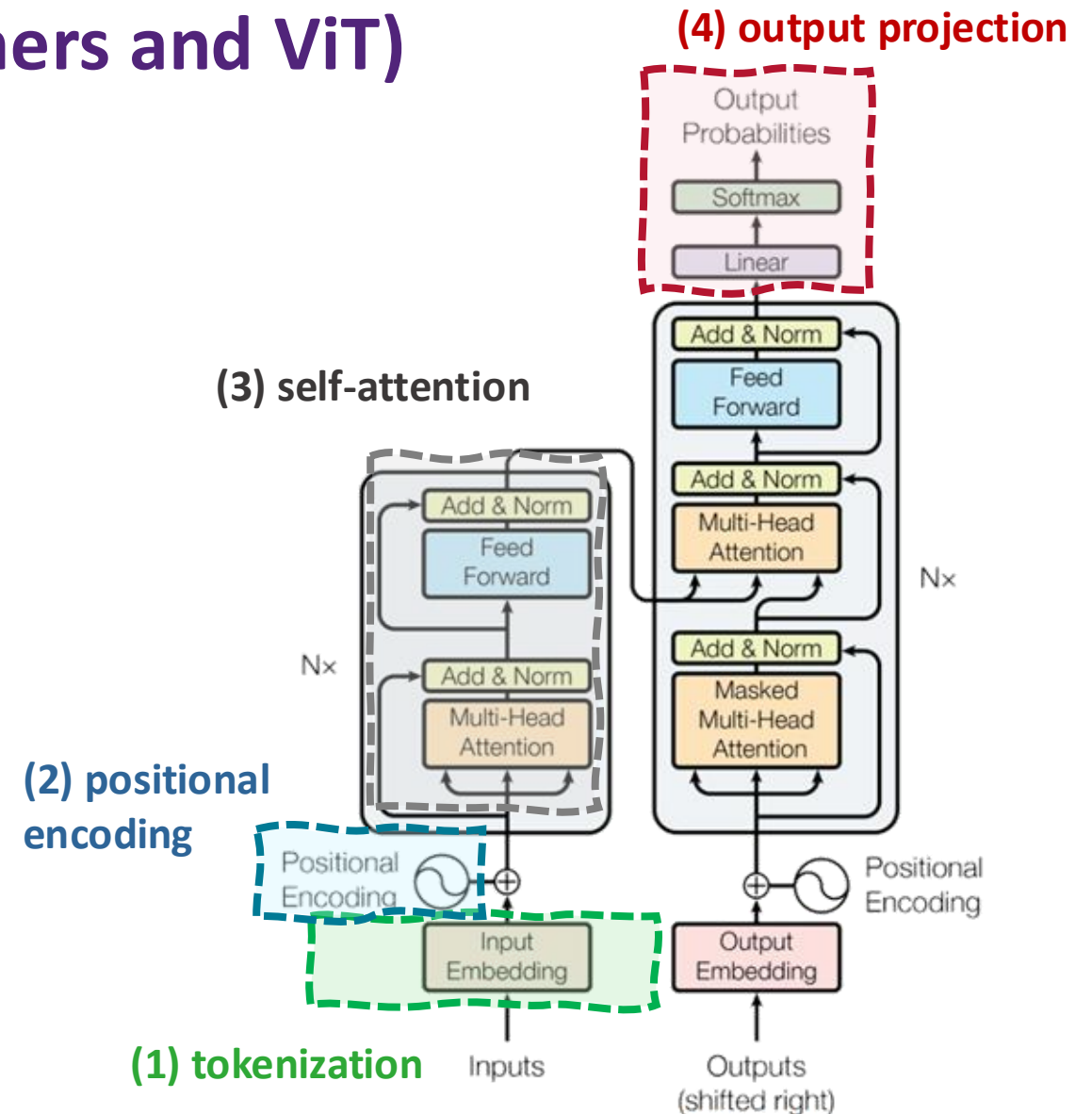
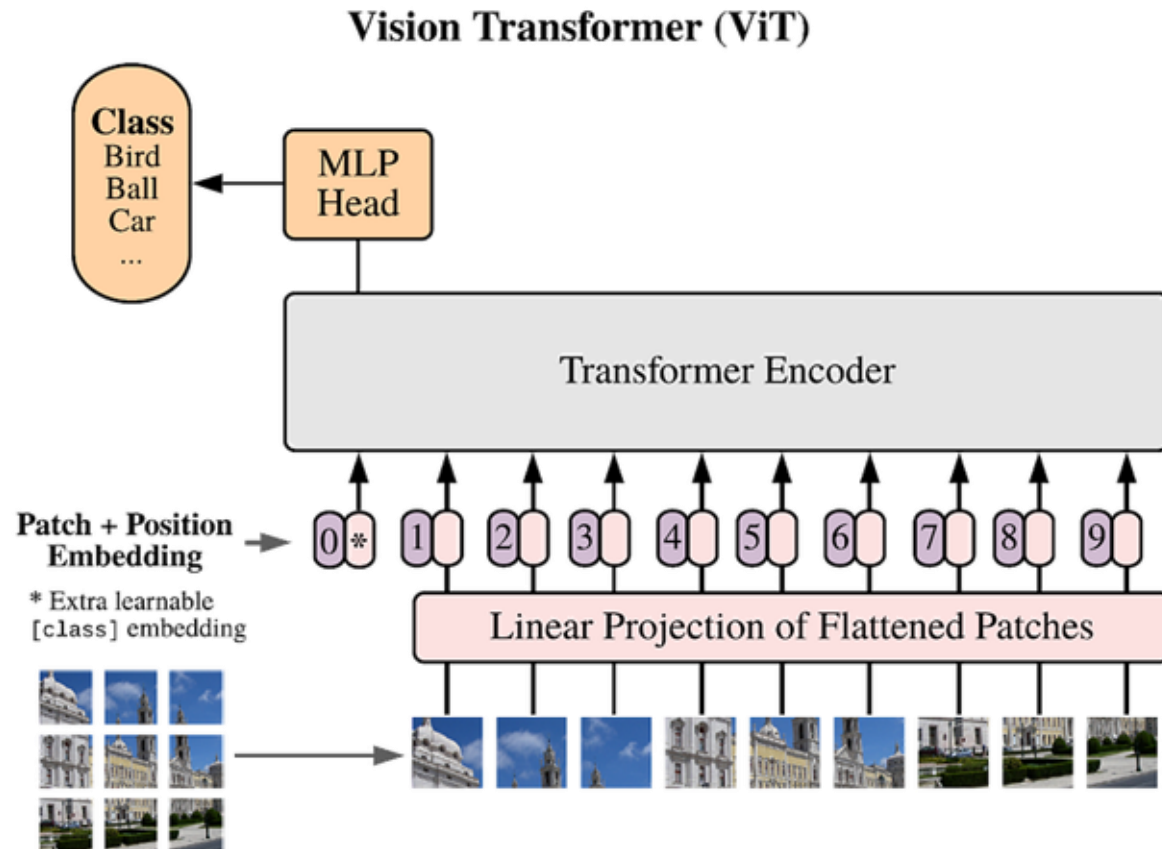
School of EECS, The University of Queensland

Outline

After today's class, you should be able to

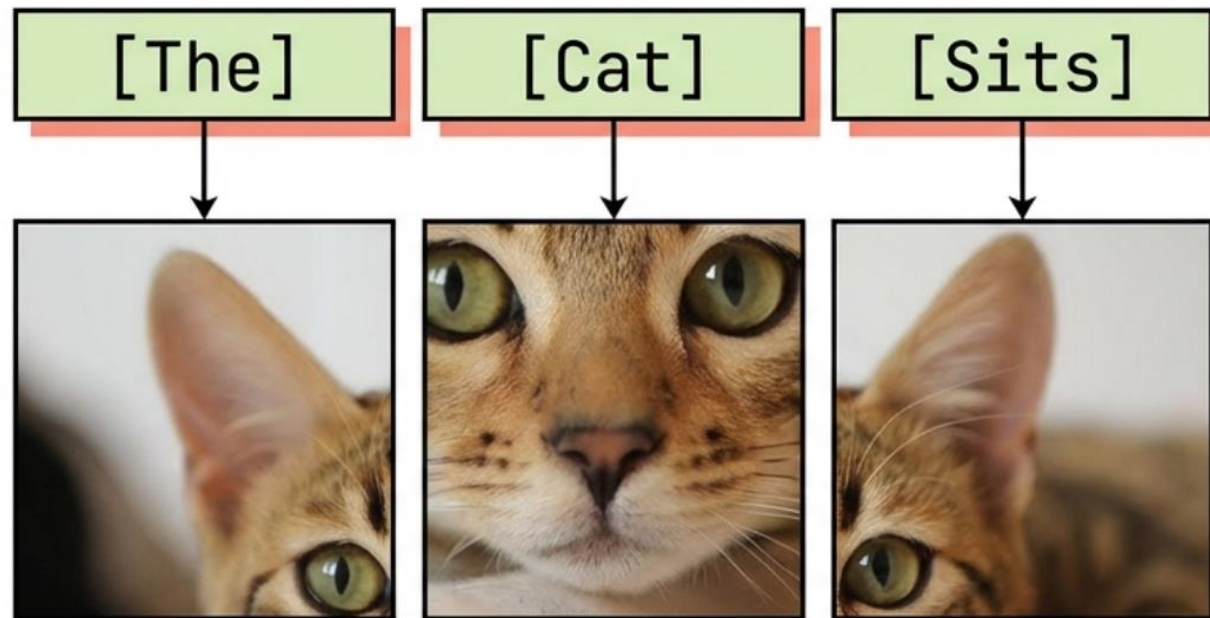
- Observe the **scaling laws** (Data, Model, Compute) applied to Transformers
- Understand the **self-supervised training** recipe and SSL Families
- Master the **DINO** and teacher-student training

Week 2 Recap... (Transformers and ViT)



Vision Transformer: From Words to Pixels

How do we feed 2D images into 1D Transformers?



- **Concept:** Serialize the 2D grid into a 1D sequence.
- **Goal:** Keep the Encoder architecture identical to NLP.

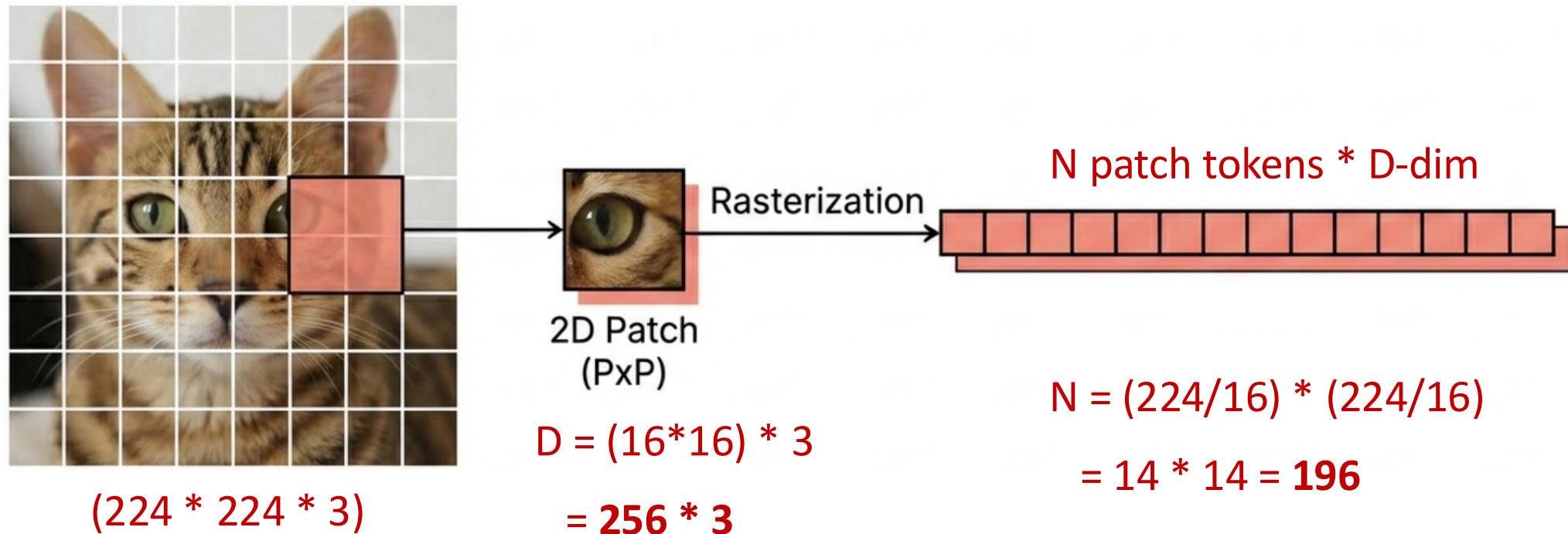
N = patches, d = dim,
 P = patch_size,
 C = channels

$$N = (H/P) \times (W/P)$$

The Patchify Operation

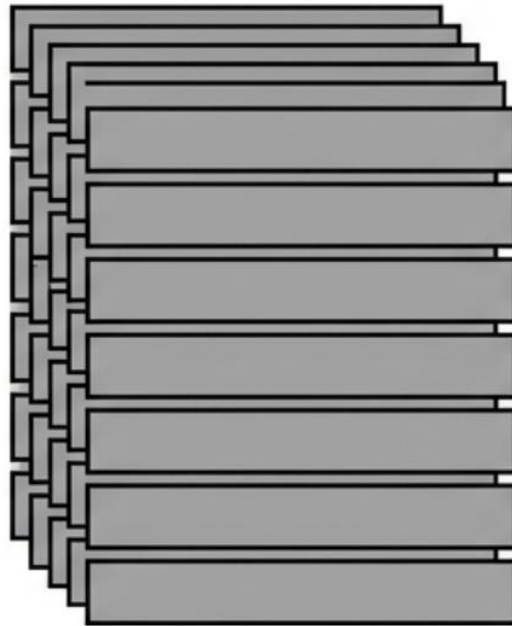
- **Grid:** Divide image HxW into fixed PxP squares.
- **Flatten:** Rasterize each square into a 1D vector.
- **Sequence:** Result is N raw vectors.

$$\text{patch_dim} = P \times P \times C$$

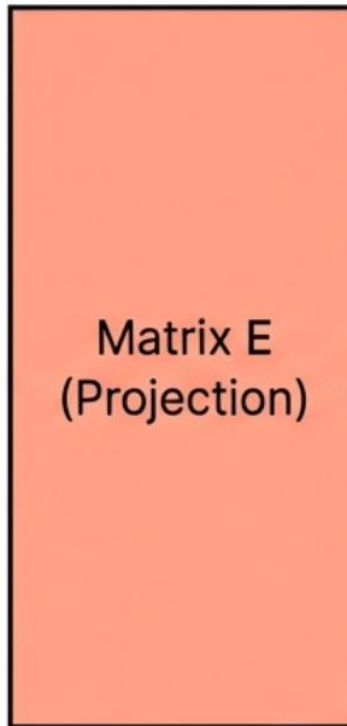


The Patchify Operation

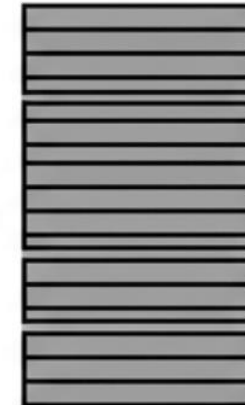
```
[pixel1_R, pixel1_G, pixel1_B,  
 pixel2_R, pixel2_G, pixel2_B,  
 ...  
 pixel256_R, pixel256_G, pixel256_B]
```



Raw Patches
(N, P^2C)



Matrix E
(Projection)



Latent Vectors
 $X (N, d)$

Compared to text embedding, any differences?

- **Input:** Raw flattened patches.
- **Map:** Learnable matrix E projects raw pixels to latent dimension d .
- **Implementation:** Conv2D (kernel= P , stride= P).

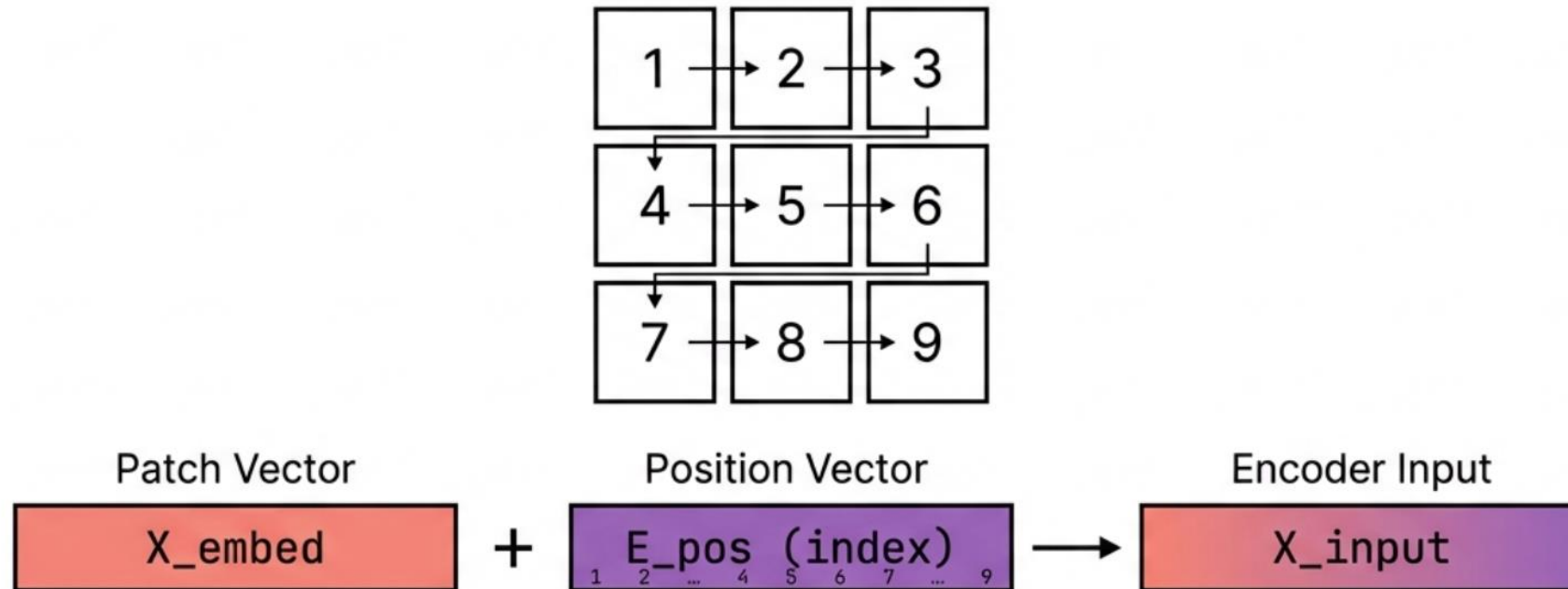
$$X = \text{Patchify}(I) \cdot E_{\text{proj}}$$

- ViT-B/16 $\rightarrow D = 768$
- ViT-L/16 $\rightarrow D = 1024$
- ViT-H/14 $\rightarrow D = 1280$

```
self.proj = nn.Conv2d(3, 768, kernel_size=16, stride=16)
```

Positional Encoding for 2D Patches

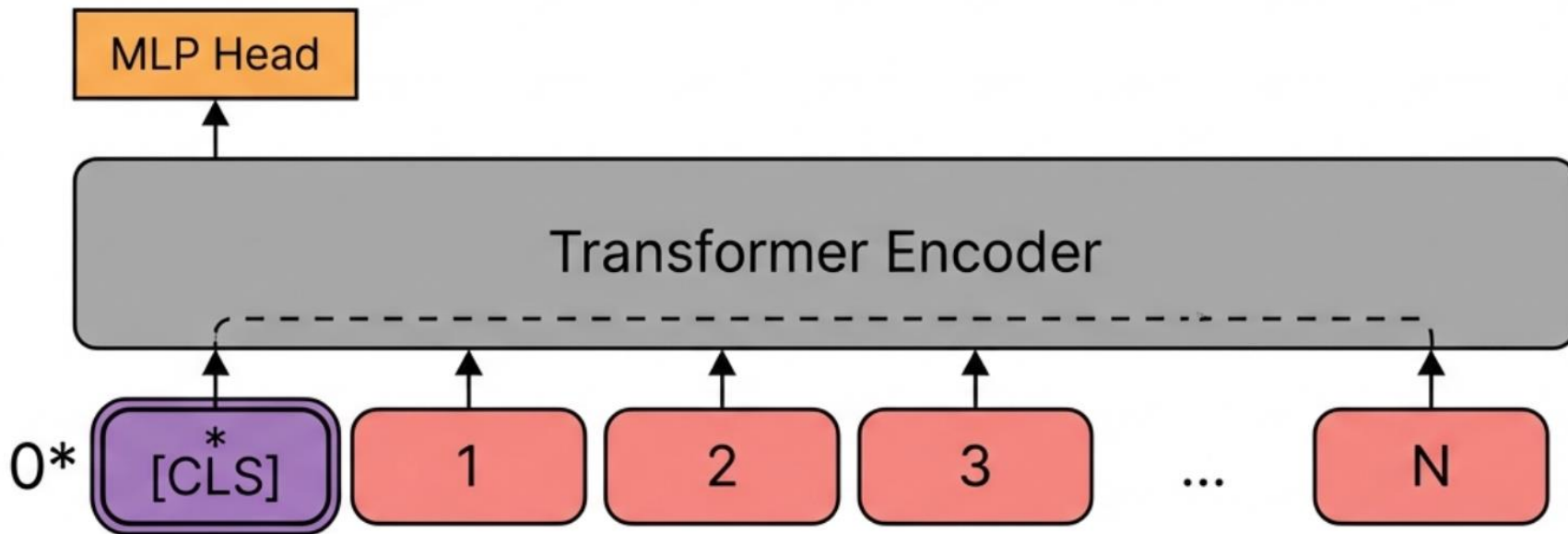
Restoring the spatial grid structure



- Problem: Flattening destroys 2D spatial info.
- Solution: Add learnable position vectors to patch embeddings.
- Format: 1D learnable parameters.

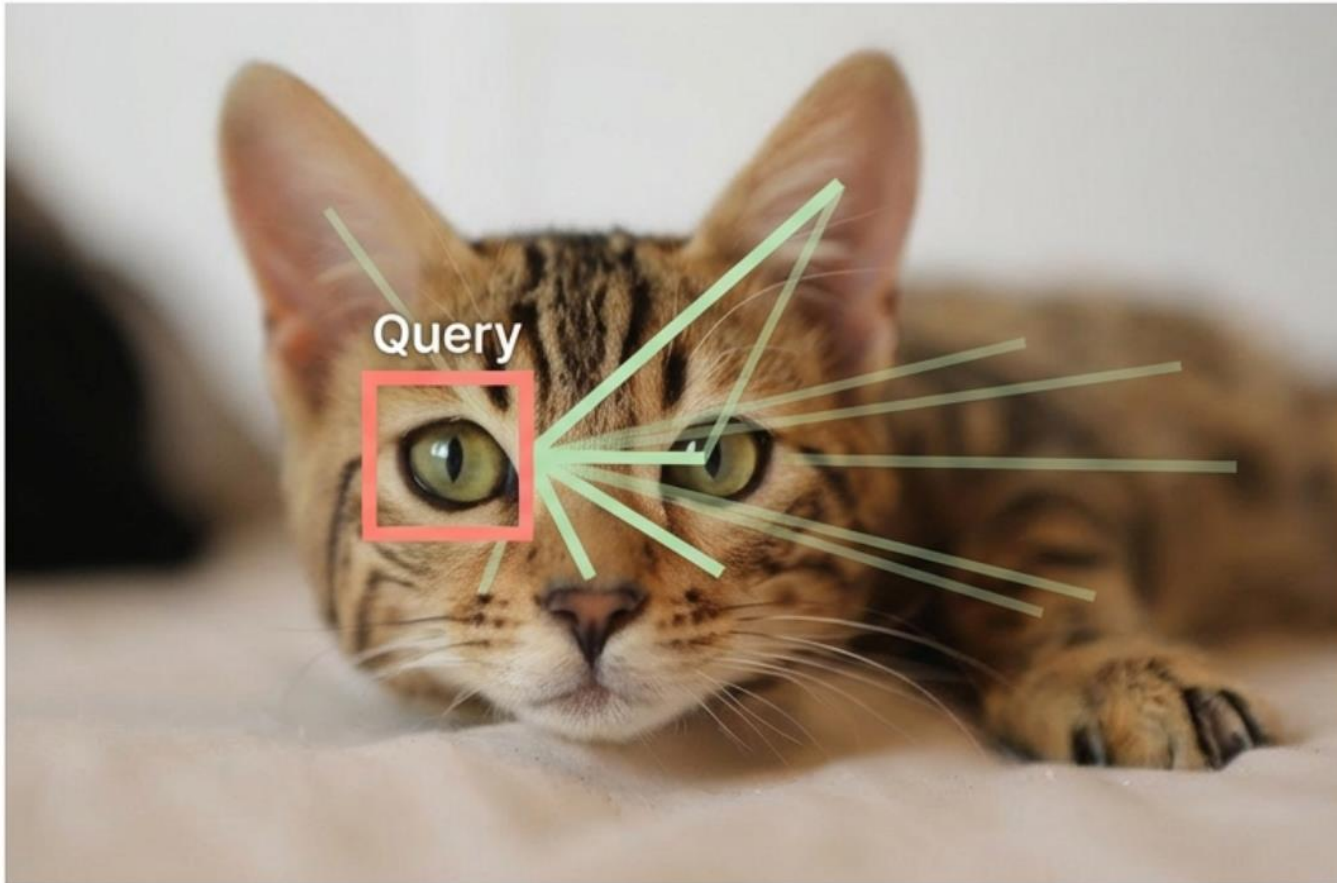
$$X_{\text{input}} = X_{\text{embed}} + E_{\text{pos}} \quad E_{\text{pos}} \in \mathbb{R}^{197 \times D}$$

The [CLS] Token and Pooling



Role: [CLS] aggregates global image information for the classification head.

Attention in Vision

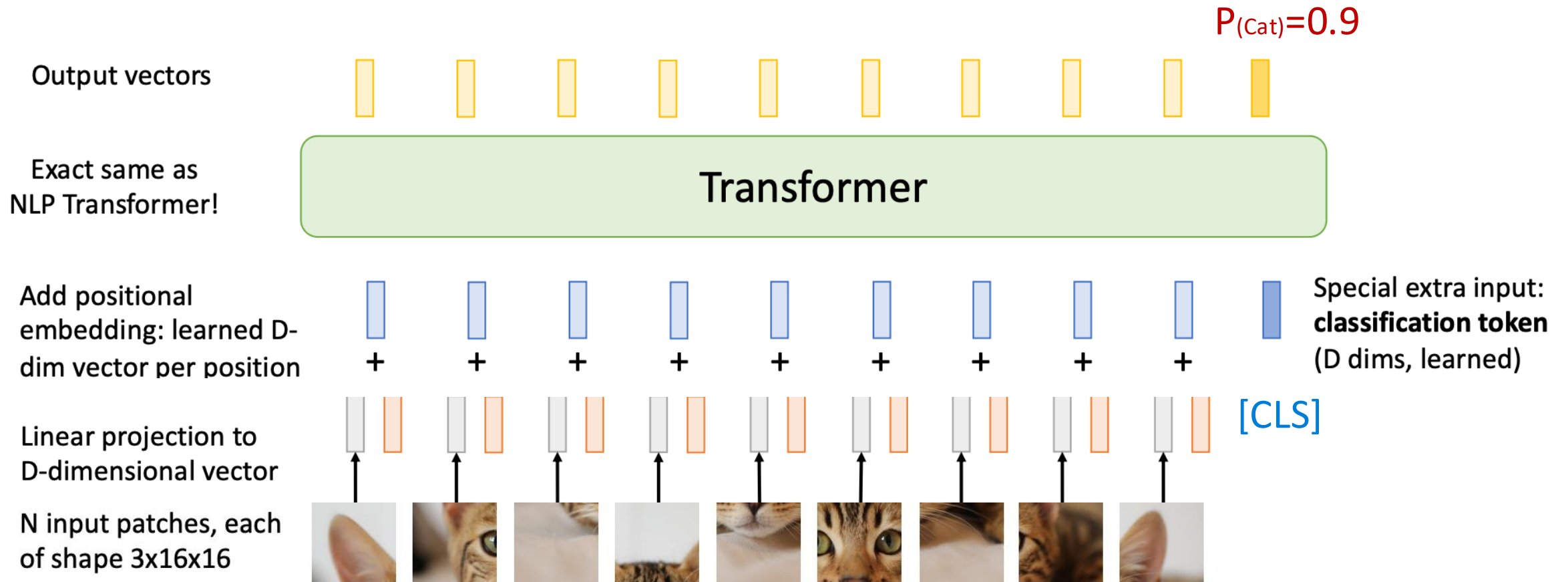


- **Mechanism:** Query patch compares with Key patches (all-to-all).
- **Global:** Receptive field is the entire image immediately.
- **Cost:** Quadratic $O(N^2)$.

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

Vision Transformer

Can we do more than classification?



DETR: Transformer for Object Detection

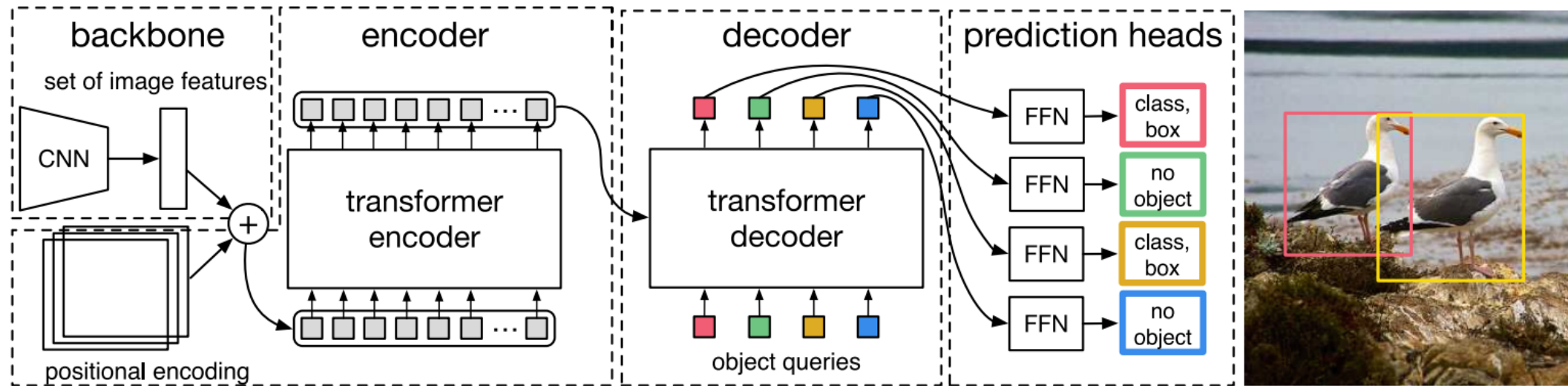
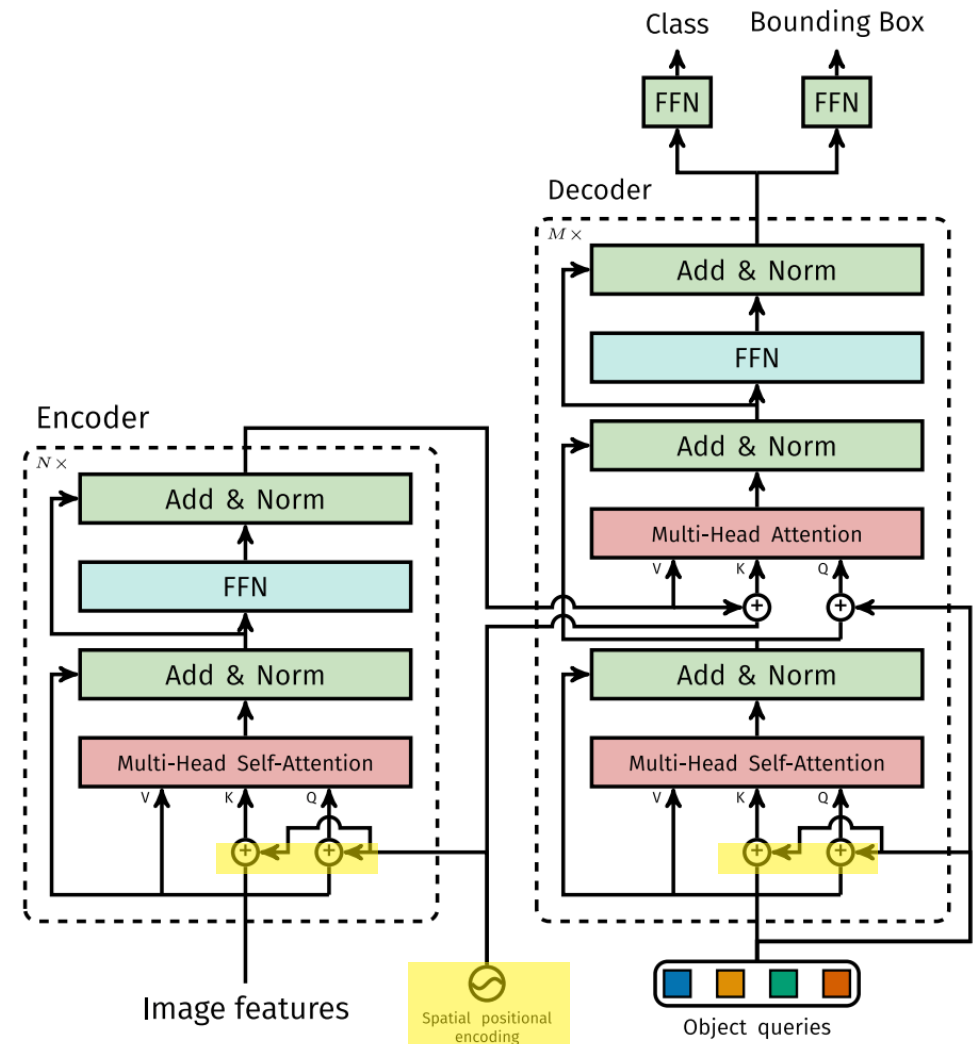


Fig. 2: DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call *object queries*, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a “no object” class.

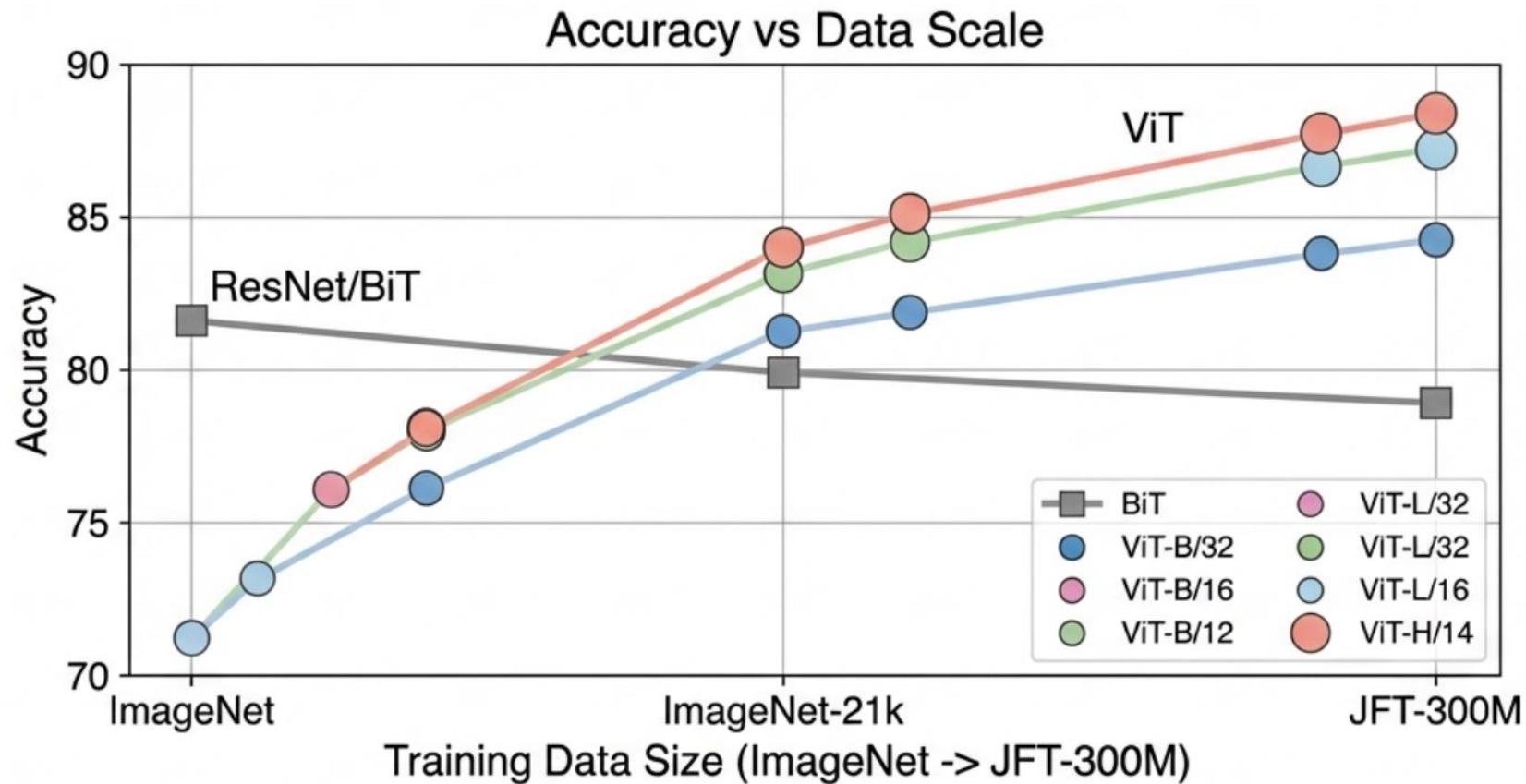
DETR: Transformer for Object Detection

- The encoder uses fixed sine positional encodings added to every self-attention layer. The x and y axes are encoded independently and concatenated.

spatial pos. enc. encoder	spatial pos. enc. decoder	output pos. enc. decoder	AP	Δ	AP ₅₀	Δ
none	none	learned at input	32.8	-7.8	55.2	-6.5
sine at input	sine at input	learned at input	39.2	-1.4	60.0	-1.6
learned at attn.	learned at attn.	learned at attn.	39.6	-1.0	60.7	-0.9
none	sine at attn.	learned at attn.	39.3	-1.3	60.3	-1.4
sine at attn.	sine at attn.	learned at attn.	40.6	-	61.6	-



Vision Transformer: Scaling up



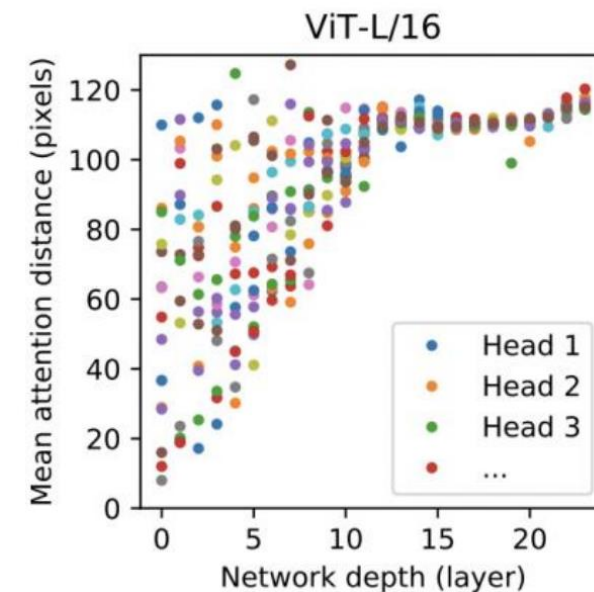
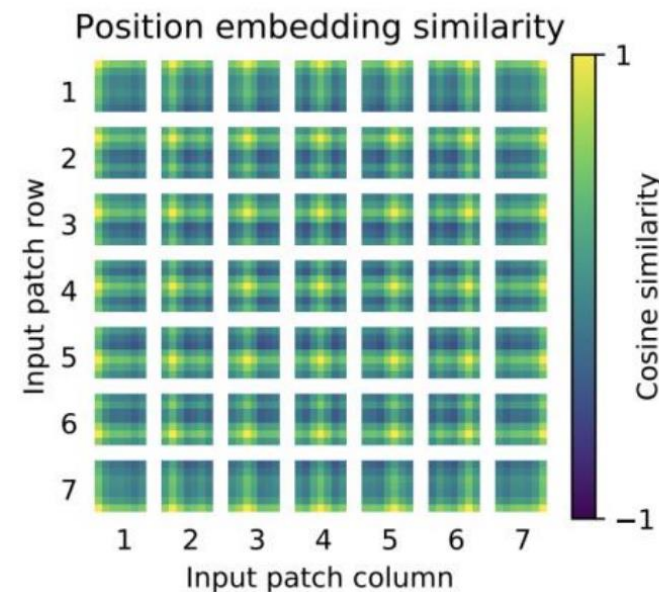
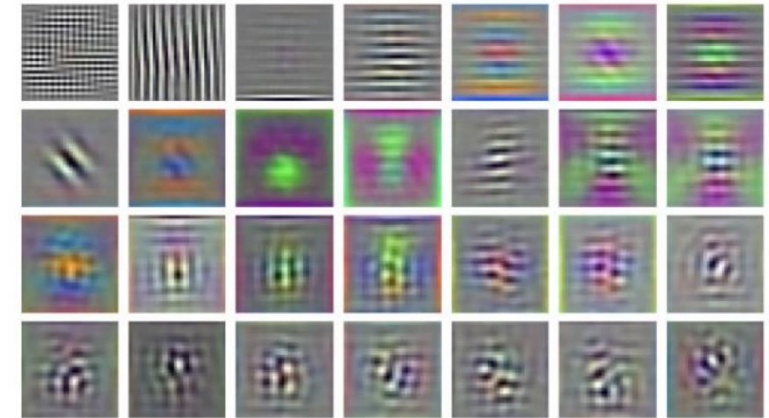
CNNs (Inductive Bias) win on small data. ViT (Scalability) wins on massive data.

Vision Transformer: Scaling up

Why ViT needs more data to perform well?

- Convolutional Neural Networks
 - Locality / Sharing weights
- Transformers
 - None!
 - Has to learn locality and dependency from **a lot lot lot of data**

RGB embedding filters
(first 28 principal components)



LLM Transformer: Scaling up

Wait... But how much we need?? -> *Scaling Law*

argue: model size (N) should grow **faster** than the size of the training set (D) for a given increase of computational budget.

$$N \propto C_{\min}^{0.73} \quad D \propto C^{0.24+0.03} = C^{0.27}$$

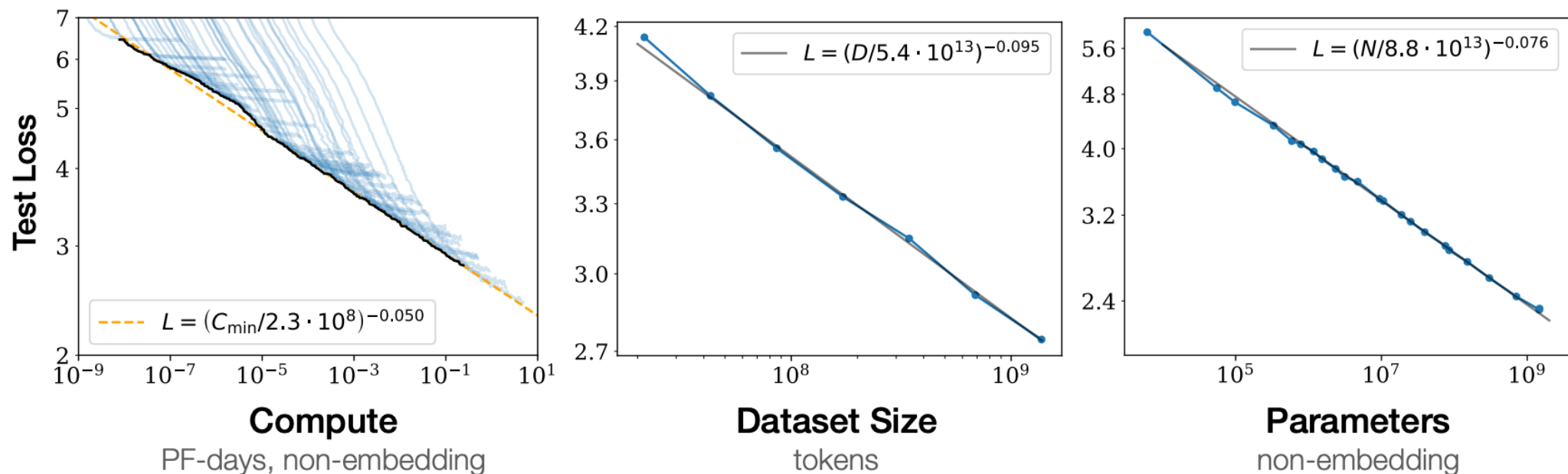


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

GPT-3, $N=175B$, $D=300B$

LLM Transformer: Scaling up

(Kaplan's suggestion)

Given 1B model, 100B tokens; now you want to scale up **10X**;

- Then it is suggested to increase the model size (N)

$$N \times 10^{0.73} \approx 5.4 \times$$

- And collect more training data

$$D \times 10^{0.27} \approx 1.9 \times$$

*Instead of 1B model + **1000B** training tokens*

$$N \propto C_{\min}^{0.73}$$

$$D \propto C^{0.24+0.03} = C^{0.27}$$

LLM Transformer: Scaling up

(Hoffman's suggestion)

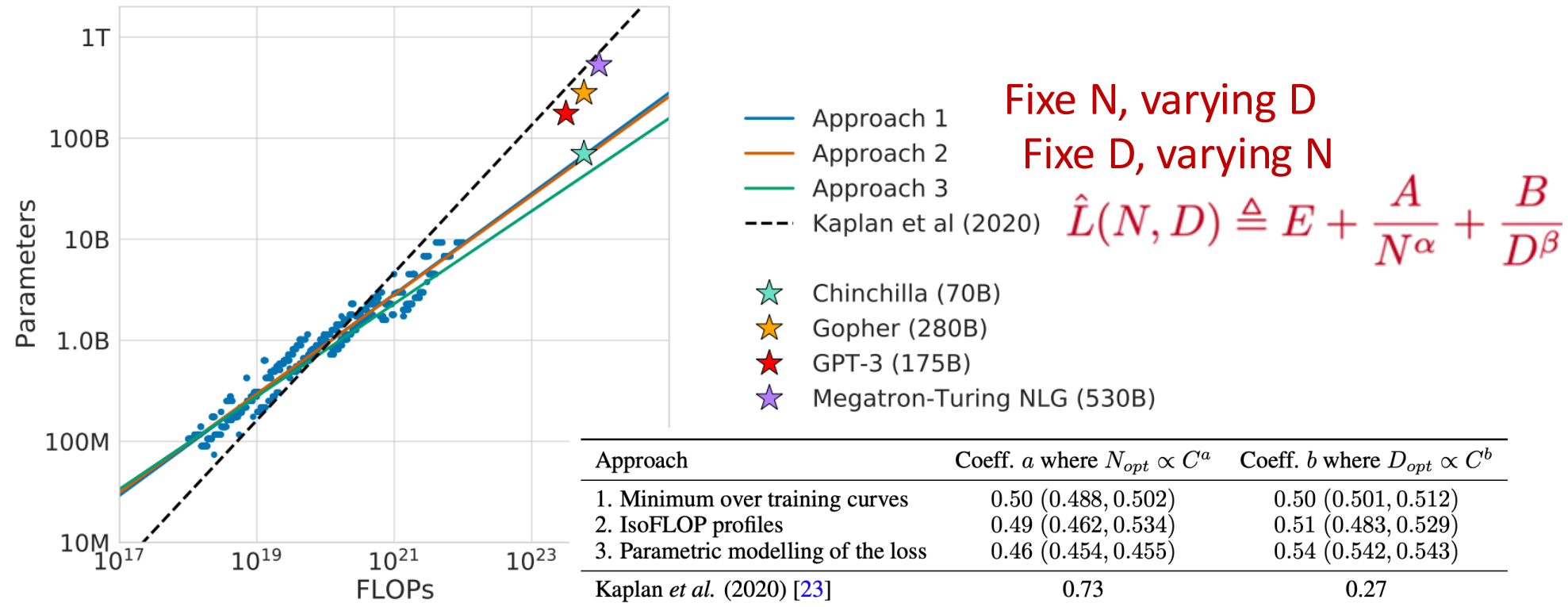


Figure 1: Overlaid predictions. We overlay the predictions from our three different approaches, along with projections from [23]. We find that all three methods predict that current large models should be substantially smaller and therefore trained much longer than is currently done. In Figure A3, we show the results with the predicted optimal tokens plotted against the optimal number of parameters for fixed FLOP budgets. ***Chinchilla* outperforms *Gopher* and the other large models (see Section 4.2).**

LLM Transformer: Scaling up

(Hoffman's suggestion)

Given 1B model, 100B tokens; now you want to scale up 10X;

- Then it is suggested to increase the model size (N)

$$N \times 10^{0.5} \approx 3.2 \times$$

- And collect more training data

$$D \times 10^{0.5} \approx 3.2 \times$$

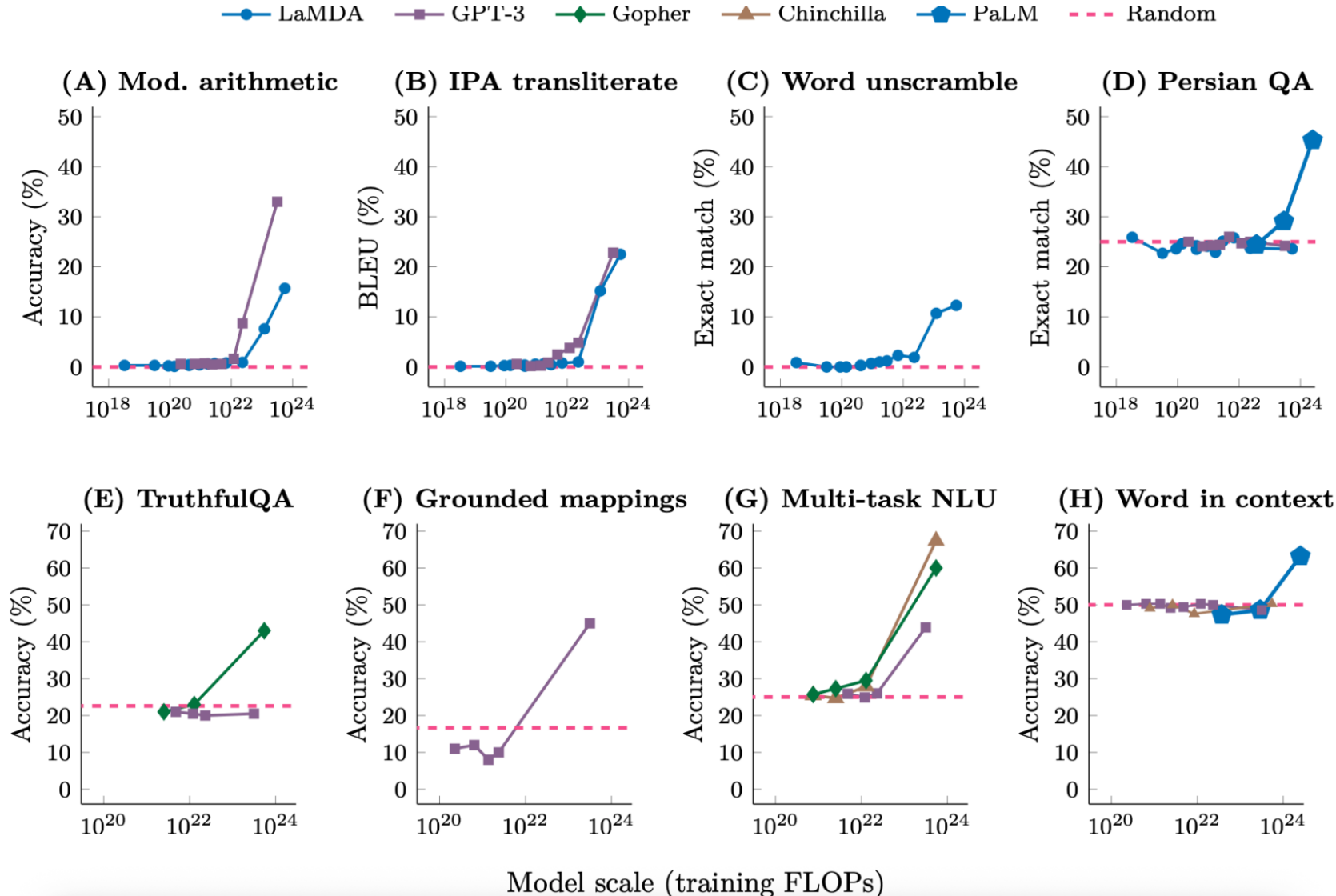
Instead of 1B model + 1000B training tokens

$$N \propto C_{\min}^{0.73}$$

$$D \propto C^{0.24+0.03} = C^{0.27}$$

LLM Transformer: Scaling up

The “Emergent” Capacity



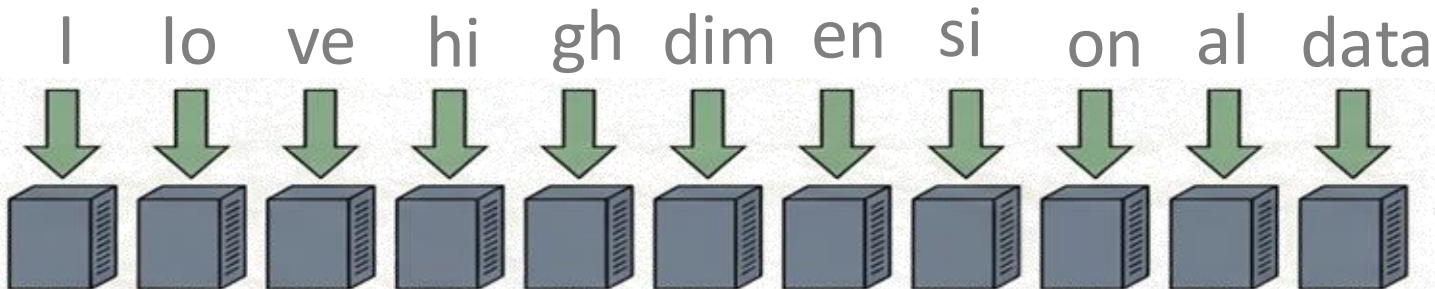
How can we transfer it to vision domain?

Scaling Needs Dense Supervision

- LLMs scale up with **dense** next-token supervision
- Vision labels are **sparse**: one label per image
- Sparse signals limit representation richness at scale
- How can we get denser learning signals for ViT

Language

$$L_{\text{text}} = \sum_t \text{CE}(p_{\theta}(x_t | x_{<t}), x_t)$$



X Vision



Y

Label

Image tag,
Bounding
boxes...

$$L_{\text{image}} = \text{CE}(p_{\theta}(\hat{y} | x), y)$$

Self-supervised Learning (SSL)

- Self-Supervised Learning (SSL) is a special type of representation learning that enables learning **good data representation** from **unlabelled** dataset.
 - Learning X from X
- Supervised learning learns from input to target that are **externally** provided (human annotators)
 - Learning Y from X

$$L_{\text{text}} = \sum_t \text{CE}(p_{\theta}(x_t \mid x_{<t}), x_t)$$

Self-supervised Learning

$$L_{\text{image}} = \text{CE}(p_{\theta}(\hat{y} \mid x), y)$$

Supervised Learning

Self-supervised Learning (SSL)

► “Pure” Reinforcement Learning (**cherry**)

- The machine predicts a scalar reward given once in a while.

► **A few bits for some samples**

► Supervised Learning (**icing**)

- The machine predicts a category or a few numbers for each input
- Predicting human-supplied data
- **10→10,000 bits per sample**

► Self-Supervised Learning (**cake génoise**)

- The machine predicts any part of its input for any observed part.
- Predicts future frames in videos
- **Millions of bits per sample**



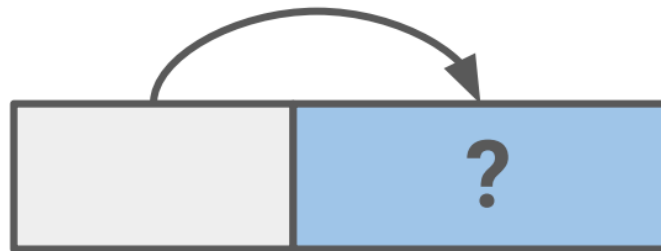
Yann LeCun's
Cake



Self-supervised Learning (SSL)

(self-prediction) Learning X_{red} from X_{blue}

- Given an individual data sample, the task is to predict one part of the sample given the other part.
- The part to be predicted **pretends** to be *missing*

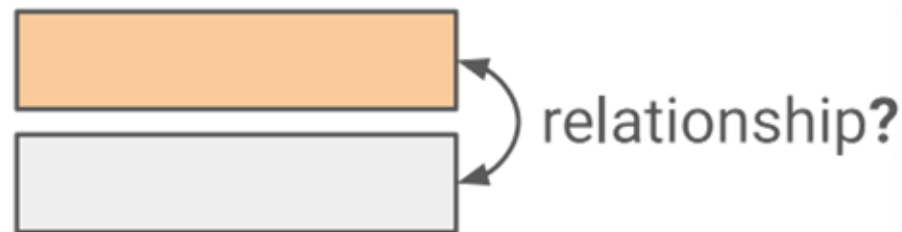


“Intra-sample” prediction

Self-supervised Learning (SSL)

(contrastive learning) Learning X_j from X_i

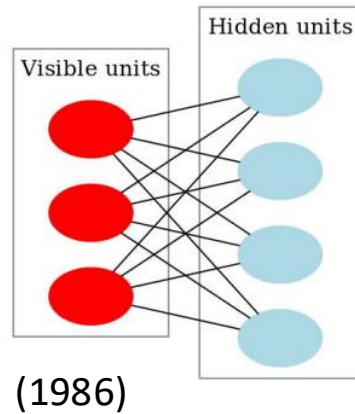
- Given multiple data samples, the task is to predict the **relationship** among them.
- The multiple samples can be selected from the dataset based on some known logics (e.g. the order of words),
- or fabricated by altering the original version (augmentation)



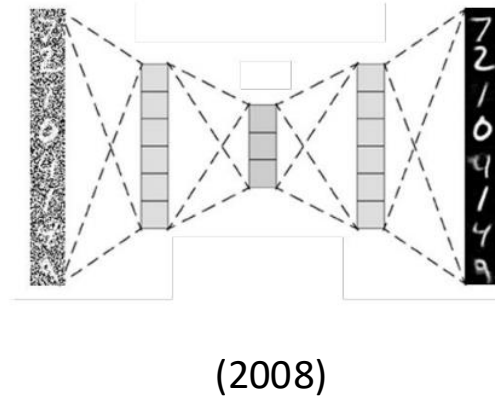
"Inter-sample" prediction

Self-supervised Learning (SSL)

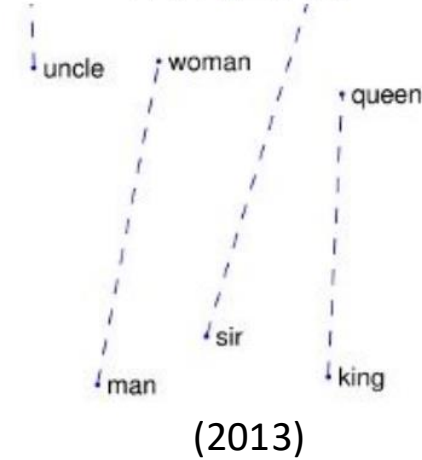
Restricted Boltzmann Machines



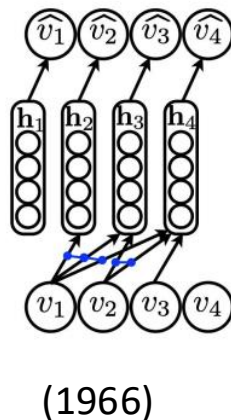
Autoencoders



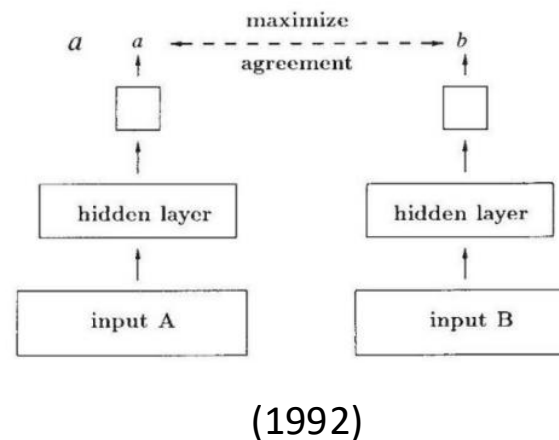
Word2Vec



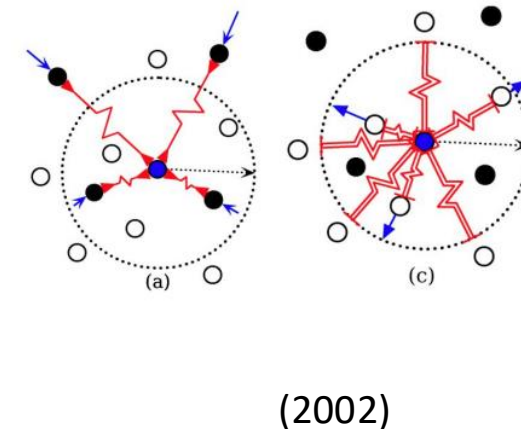
Autoregressive Modeling



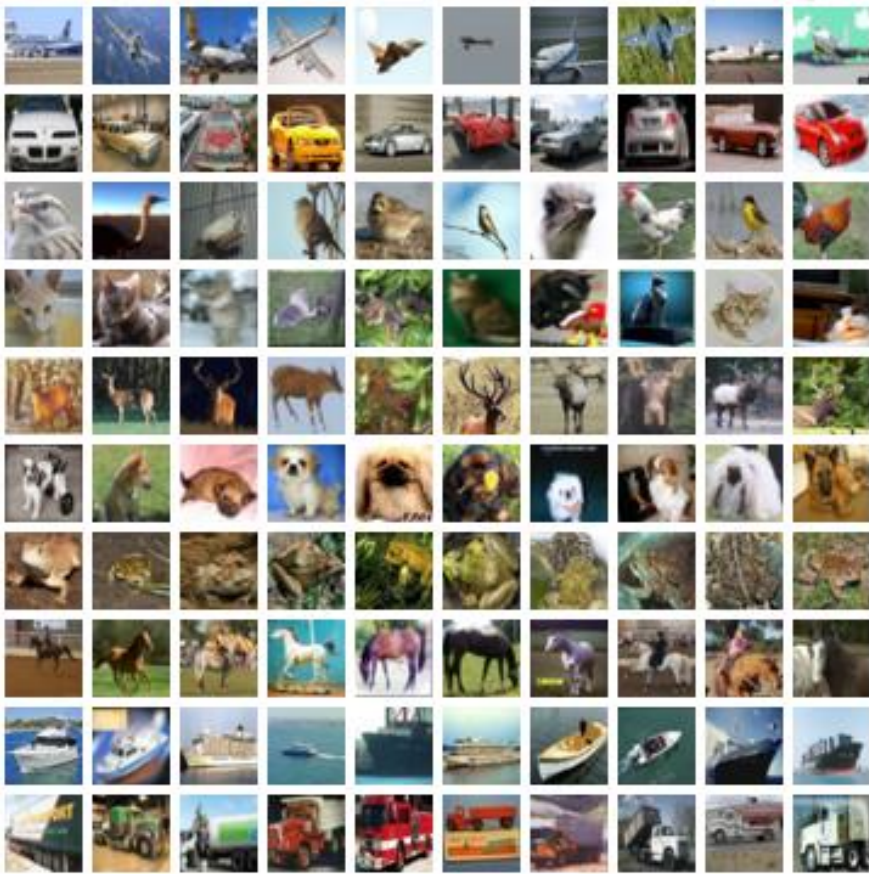
Siamese networks



Multiple Instance/Metric Learning



Self-supervised Learning (SSL)



Encoder

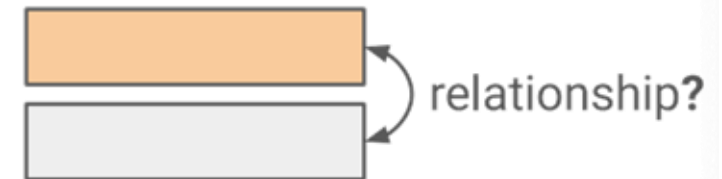
(1) Self-prediction



"Intra-sample" prediction

Pretext
Task

Not the "real"
task (like
classification) we
care about

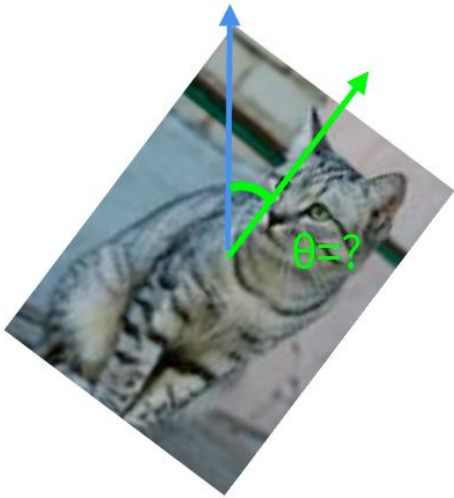


"Inter-sample" prediction

(2) Contrastive Learning

Self-Prediction Pretext Tasks

Example: learn to predict image transformations / corrupted images



rotation prediction



“jigsaw puzzle”



colorization

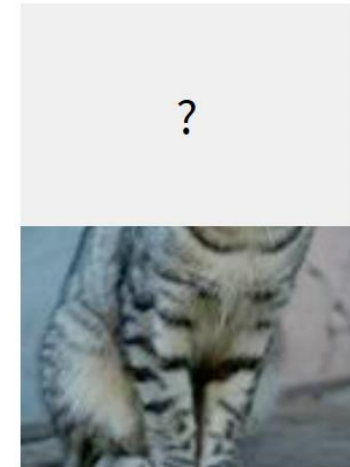


image completion

- Principle:**
1. Solving the pretext tasks allow the model to learn good features
 2. We can automatically generate “labels” for the pretext tasks

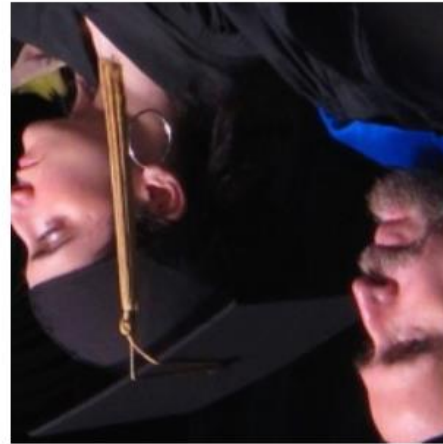
Pretext Task: predict rotations



90° rotation



270° rotation



180° rotation



0° rotation

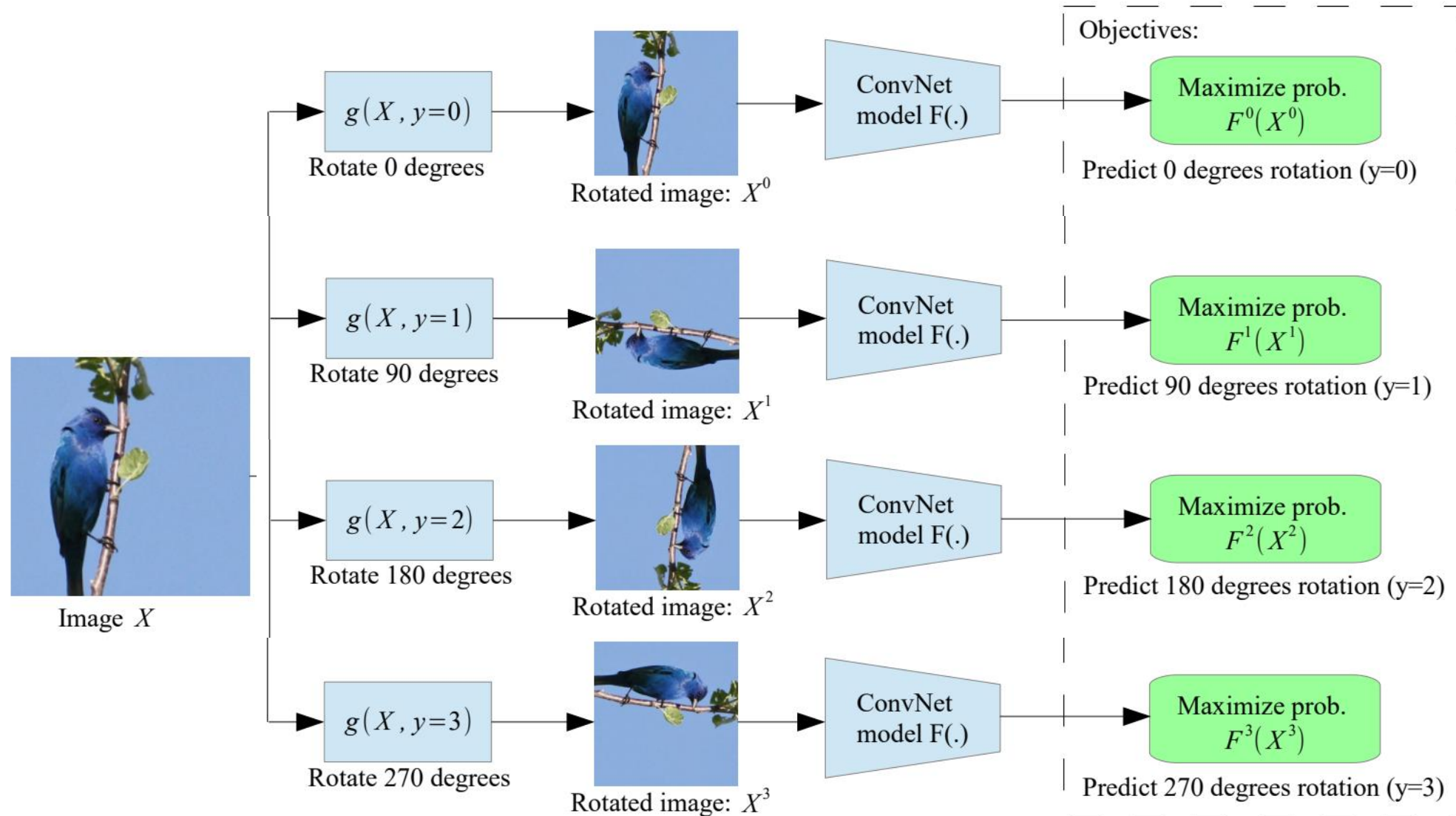


270° rotation

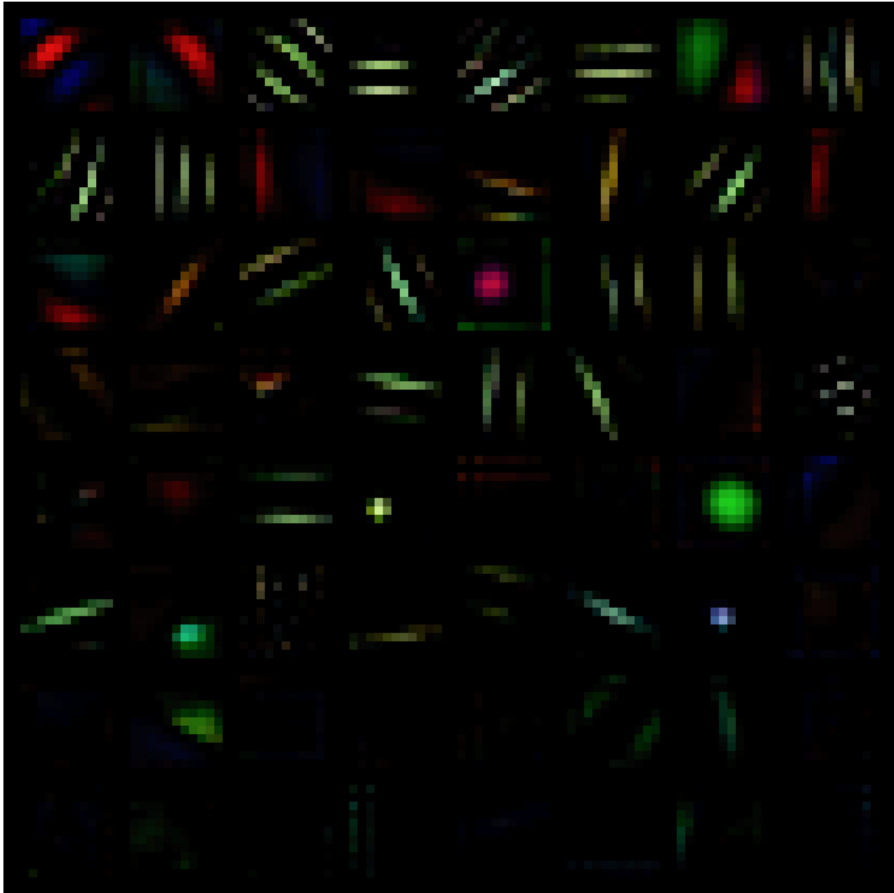
Label: y

Hypothesis: a model could recognize the **correct rotation** of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

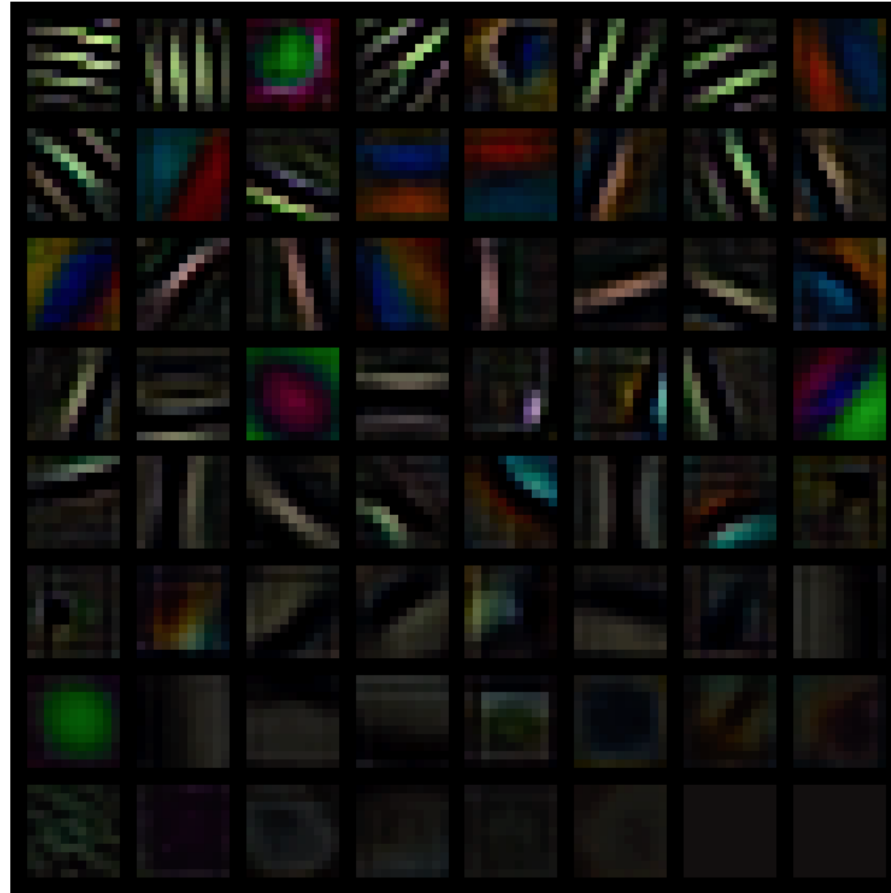
Pretext Task: predict rotations



Pretext Task: predict rotations



(a) Supervised

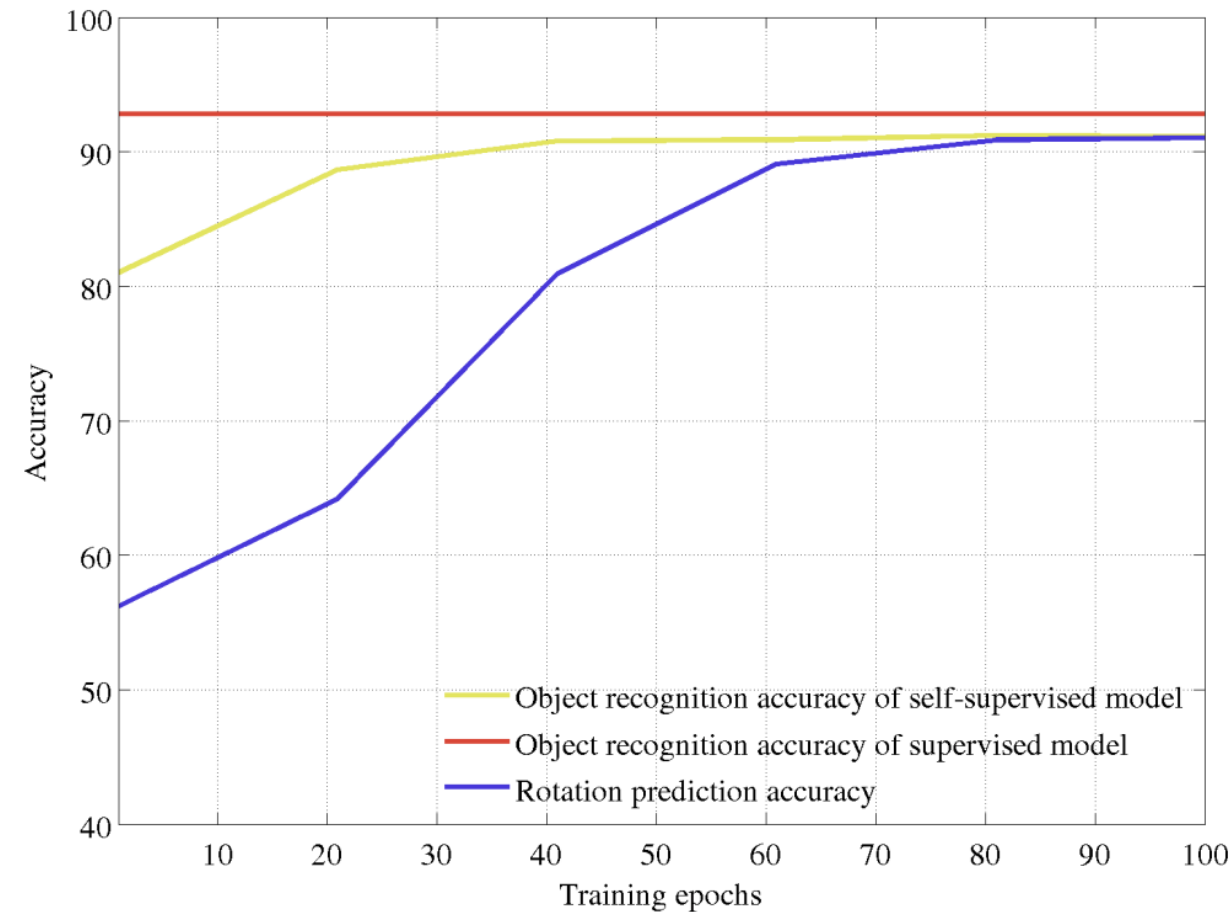


(b) Self-supervised to recognize rotations

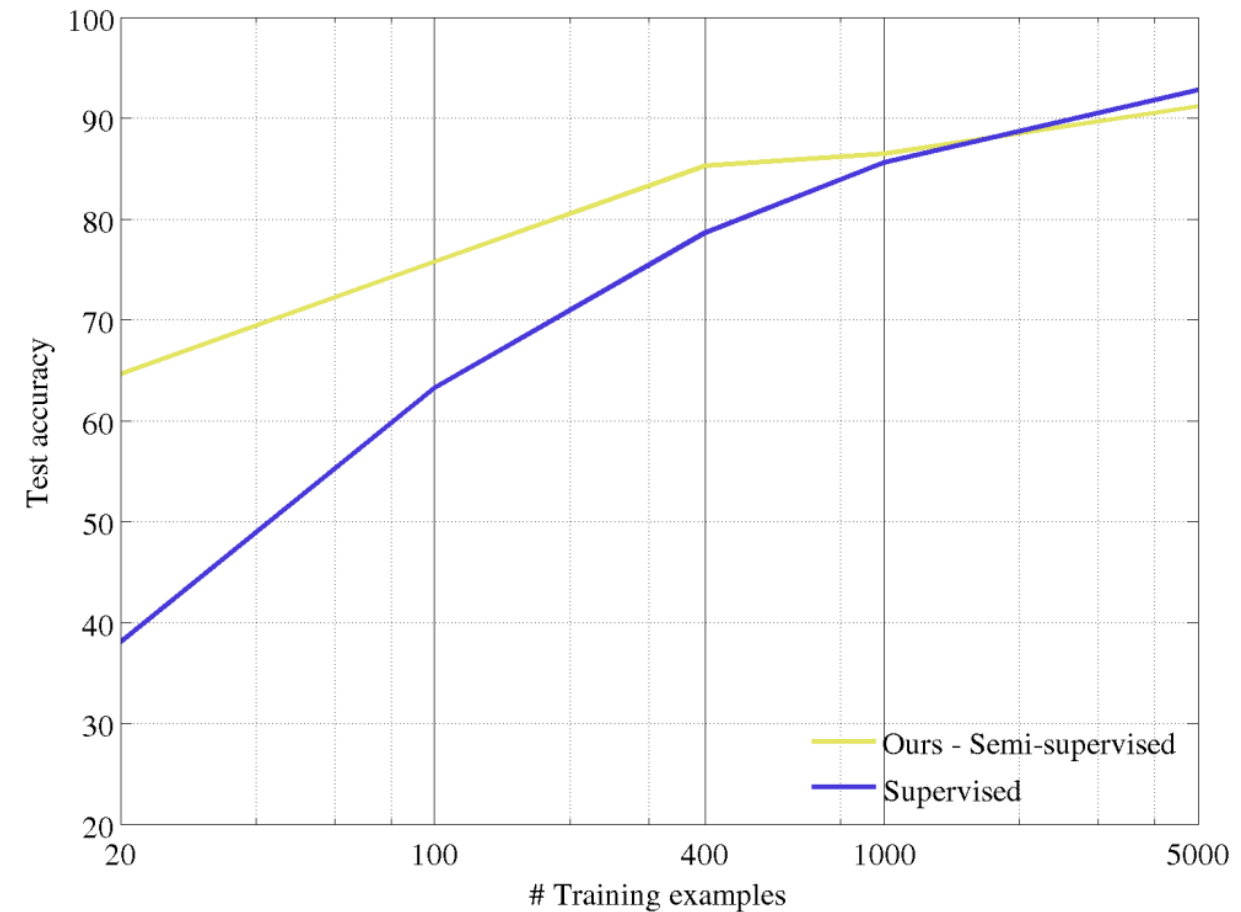
Filters learned by SSL are mostly oriented edge filters on various frequencies

Pretext Task: predict rotations

Any observation?

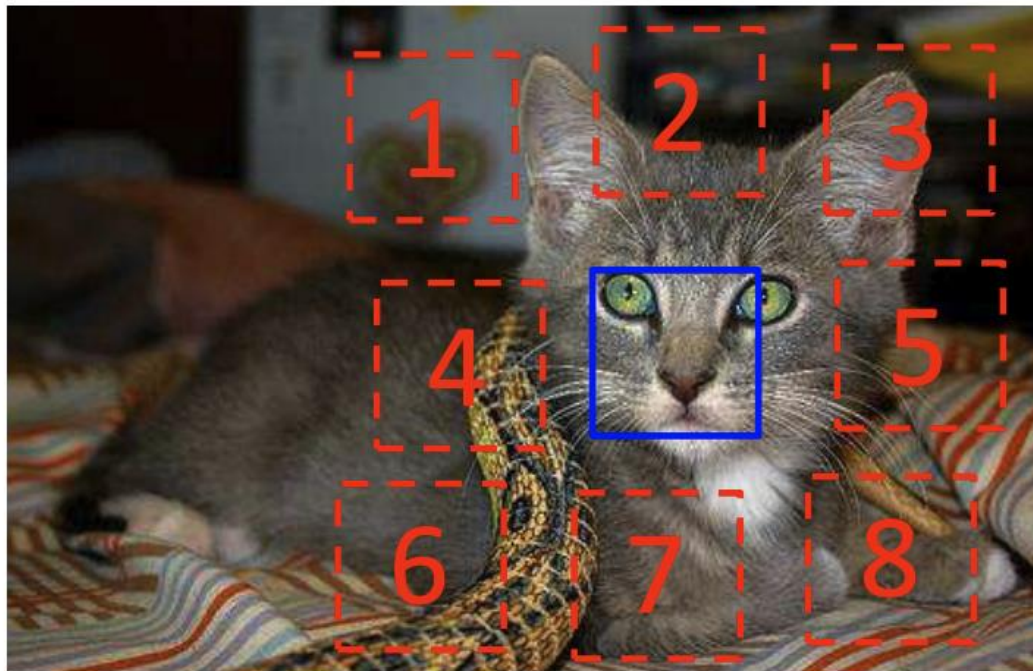


(a)



(b)

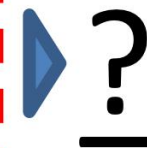
Pretext Task: predict location



$$X = \left(\begin{array}{c} \text{Kitten face patch} \\ \text{Kitten ear patch} \end{array} \right); Y = 3$$

Can you guess the spatial configuration for the two pairs of patches?

Question 1:



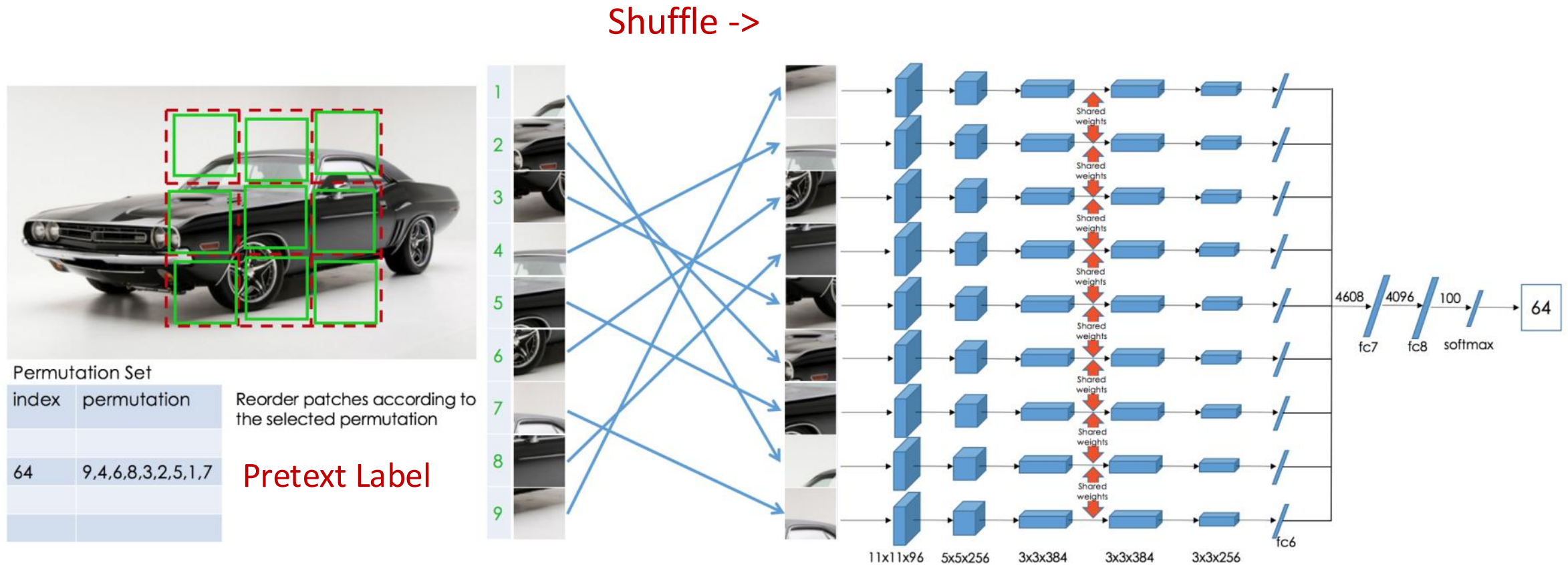
Question 2:



Answer key: Q1: Bottom right Q2: Top center

Hypothesis: a model could recognize the **neighbour patch** of an object only if it has the “visual commonsense” of what the object should look like

Pretext Task: predict location (Jigsaw Puzzles)



Pretext Task: predict location (Jigsaw Puzzles)

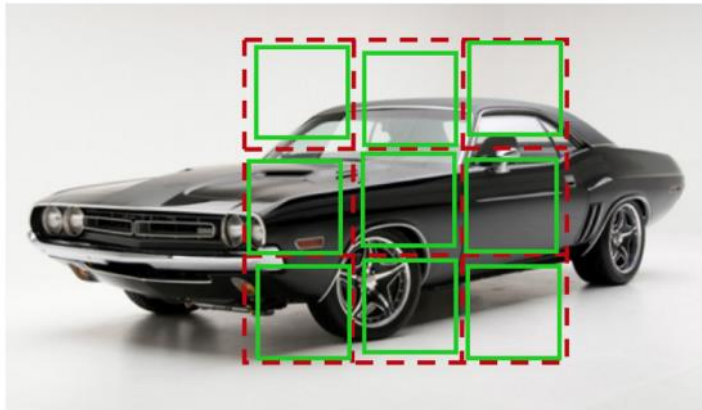


- Hypothesis:**
- **Low-level** statistics (color and texture) are different for separate instances within the same categories.
 - The Jigsaw puzzle solver learns to **ignore** such statistics when they do not help the localization of parts.

relative location

Method	Pretraining time	Supervision	Classification	Detection	Segmentation
Krizhevsky <i>et al.</i> [25]	3 days	1000 class labels	78.2%	56.8%	48.0%
Wang and Gupta[39]	1 week	motion	58.4%	44.0%	-
Doersch <i>et al.</i> [10]	4 weeks	context	55.3%	46.6%	-
Pathak <i>et al.</i> [30]	14 hours	context	56.5%	44.5%	29.7%
Ours	2.5 days	context	67.6%	53.2%	37.6%

Pretext Task: predict location (Jigsaw Puzzles)



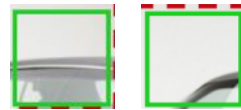
Permutation Set

index	permutation
64	9,4,6,8,3,2,5,1,7

Reorder patches according to the selected permutation

Given 9 tiles, can you guess *how many* possible permutation?

$$9! = 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 362,880$$

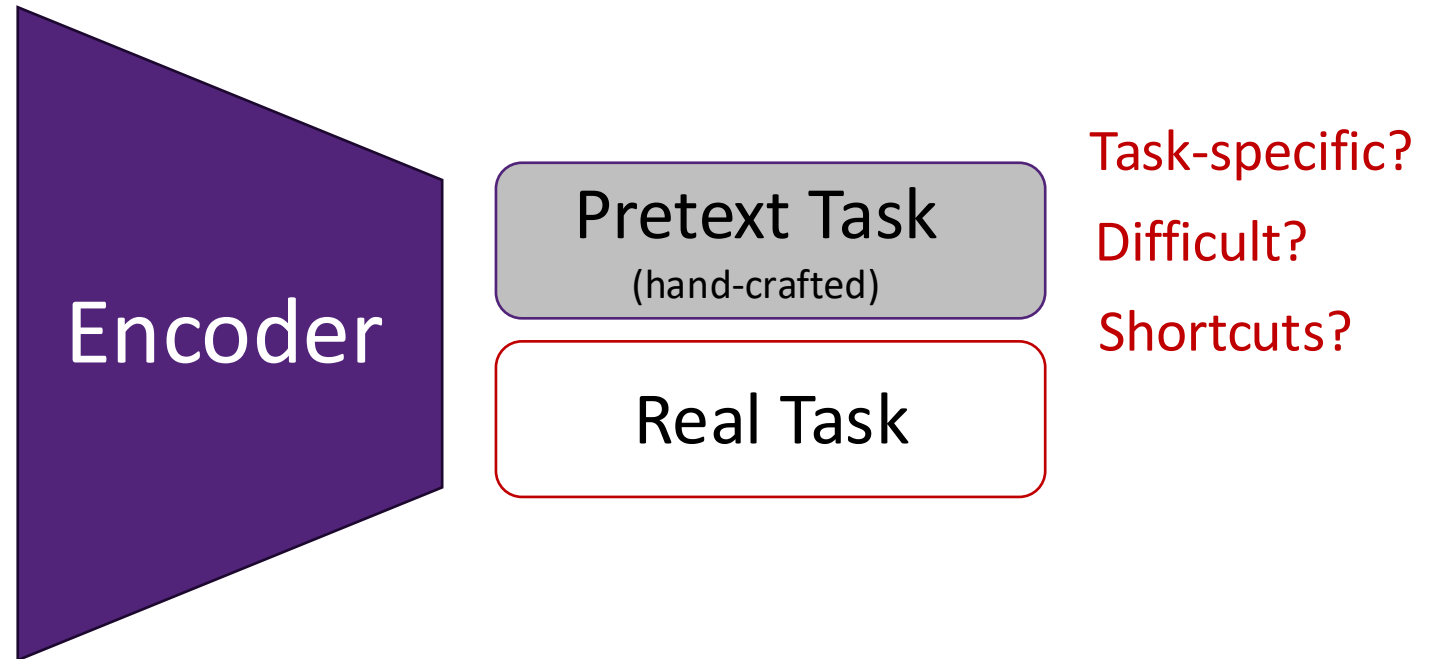
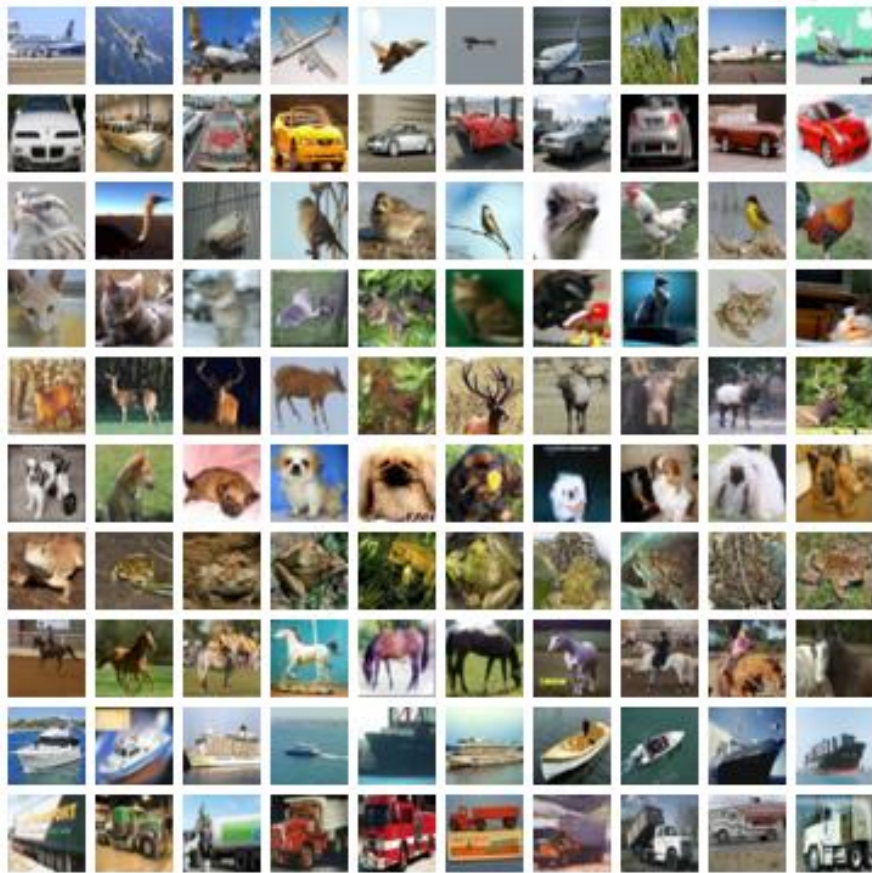


{2,1,3,4,5,6,7,8,9} Hamming Dist = 2

difficult!

Number of permutations	Average hamming distance	Minimum hamming distance	Jigsaw task accuracy	Detection performance
1000	8.00	2	71	53.2
1000	6.35	2	62	51.3
1000	3.99	2	54	50.2
100	8.08	2	88	52.6
95	8.08	3	90	52.4
85	8.07	4	91	52.7
71	8.07	5	92	52.8
35	8.13	6	94	52.6
10	8.57	7	97	49.2
7	8.95	8	98	49.6
6	9	9	99	49.7

Pretext Task: Why they don't scale?



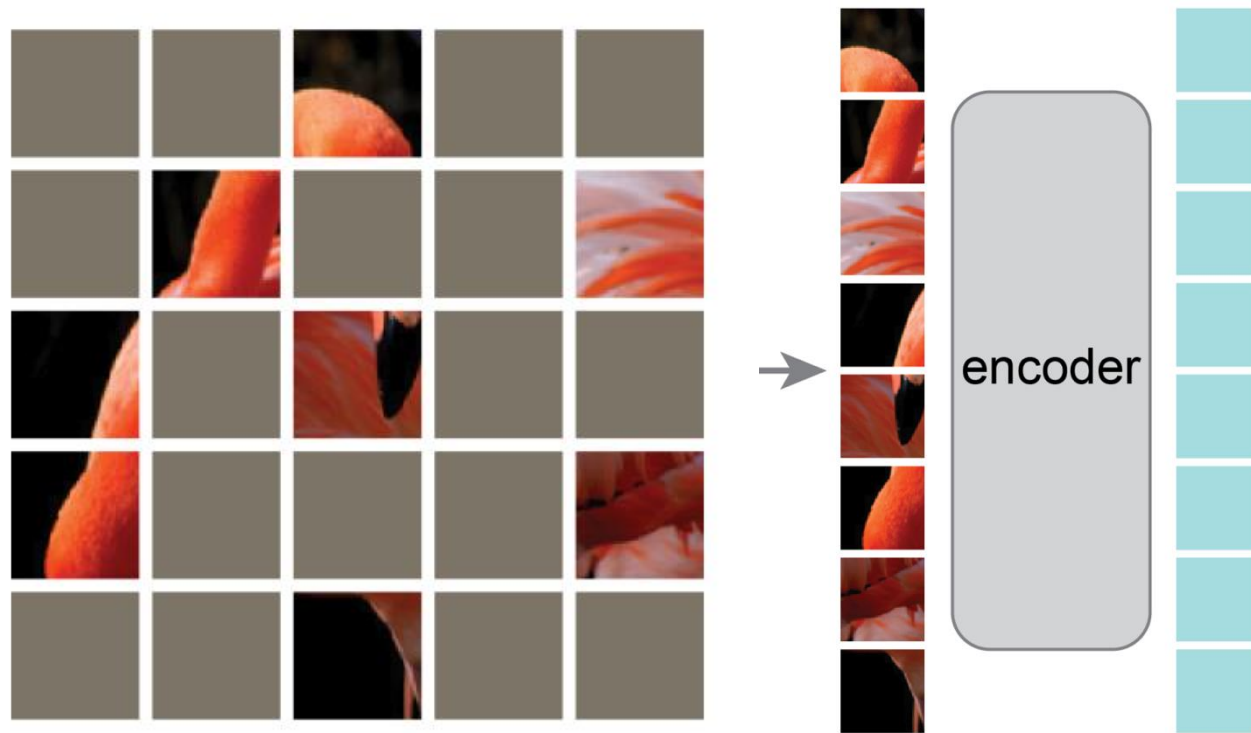
- ✓ The models are forced learn **good features** about natural images in order to solve pretext tasks
- Targets are **task-specific** (coming up individual pretext is hard) not *universal*
- Model often learn brittle **shortcuts**

Quiz Time!



Pretext Task: Masked Generation

- Reconstruction gives **dense** per-patch learning signals



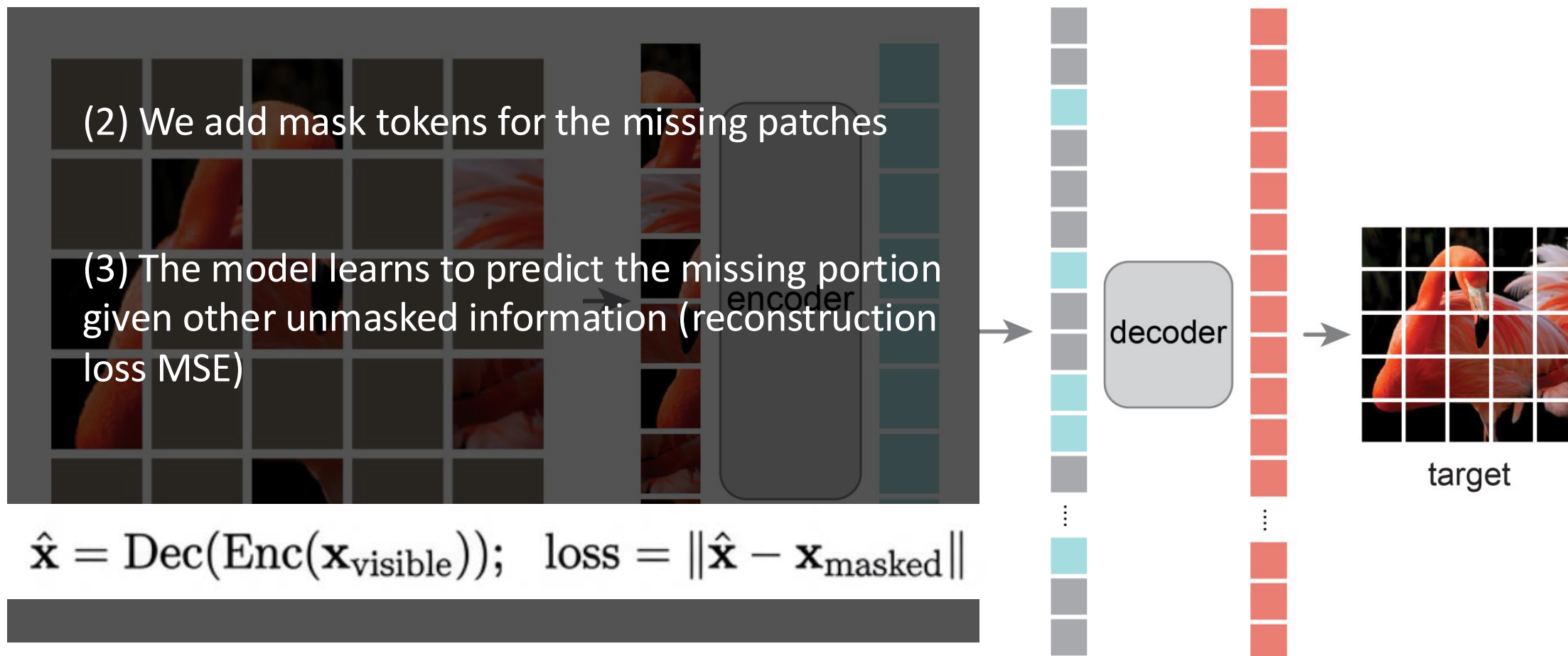
(1) Random masking

(2) Encode Visible Patches

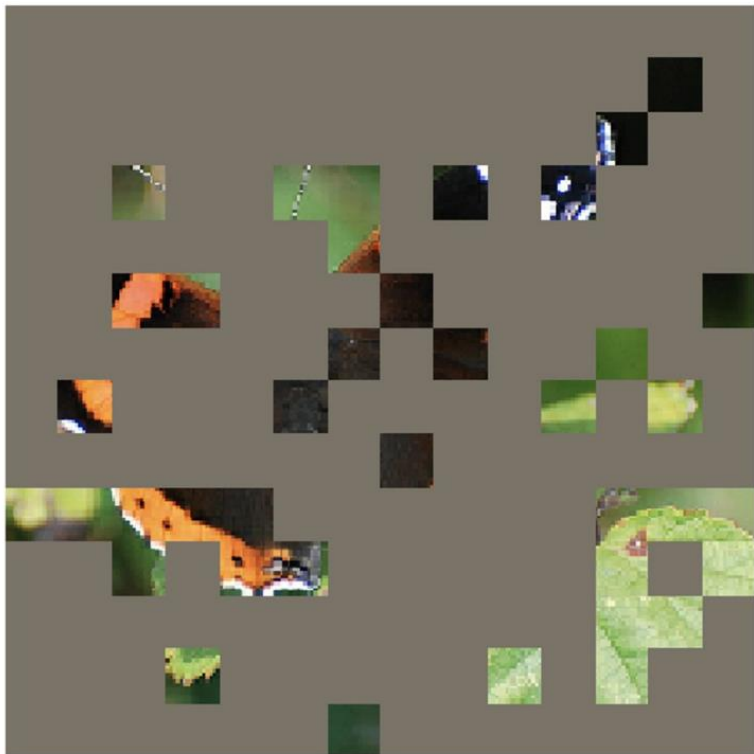
- We mask a random portion of information and pretend it is missing

Pretext Task: Masked Generation

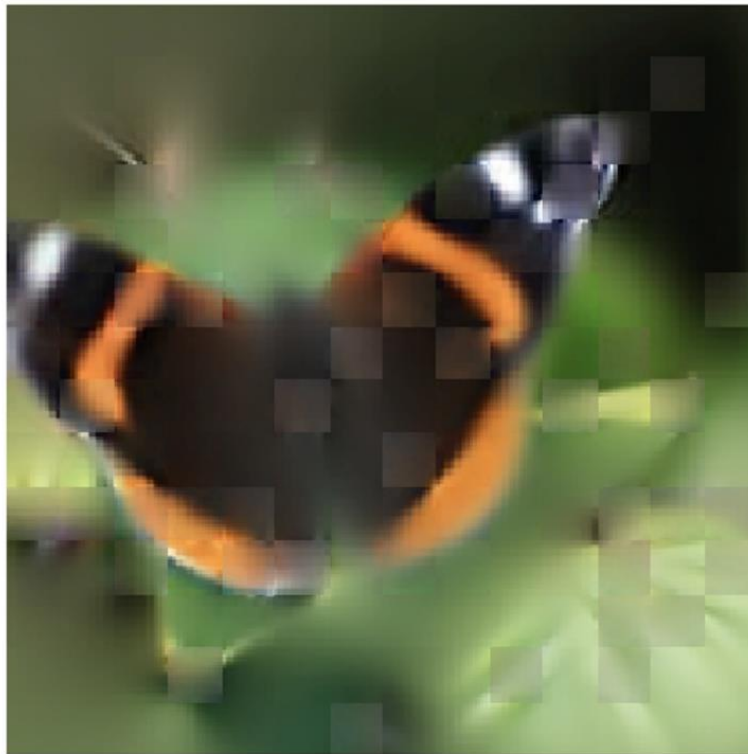
- Reconstruction gives **dense** per-patch learning signals



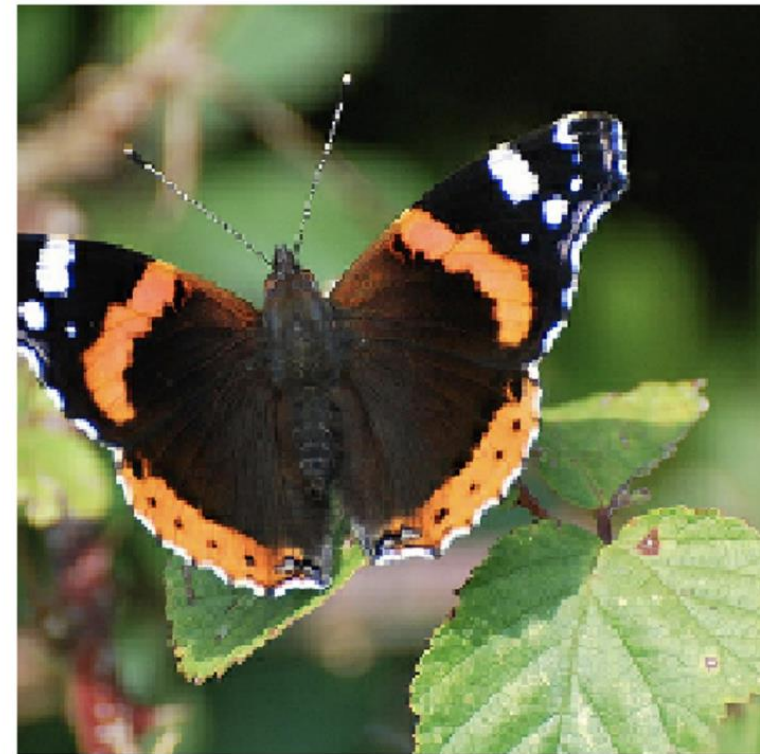
Pretext Task: Masked Generation



Masked input: 80%



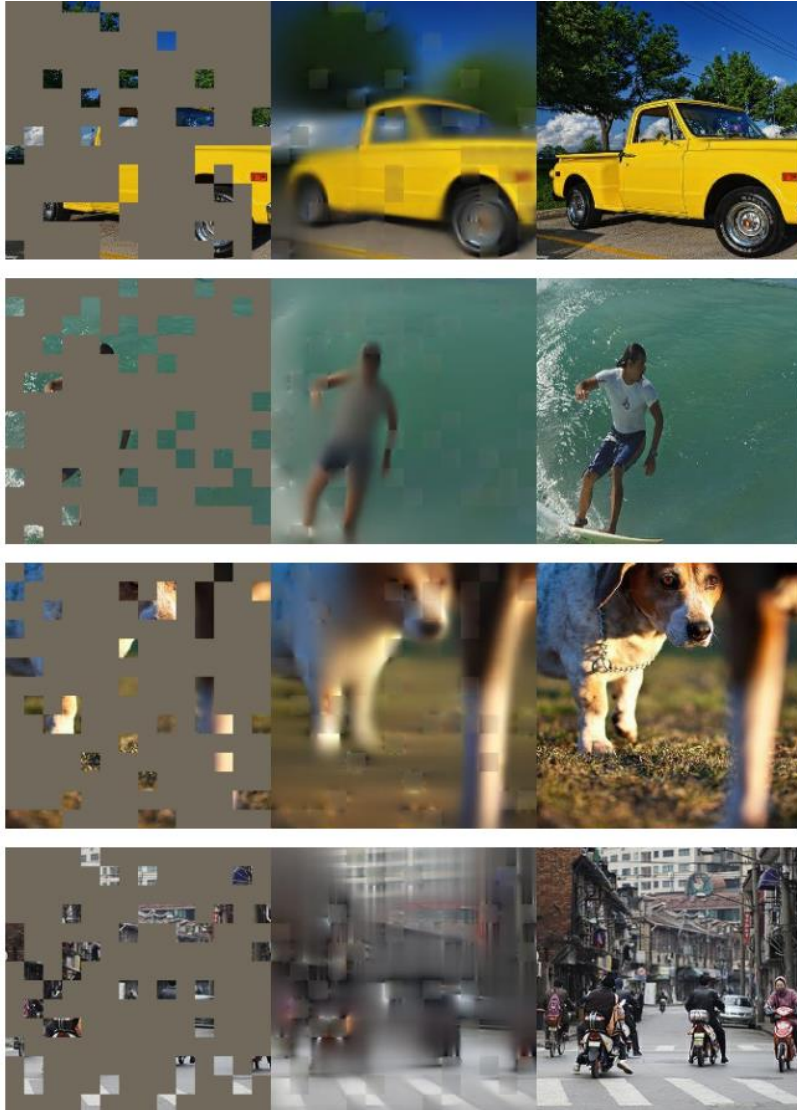
MAE's guess



Ground truth

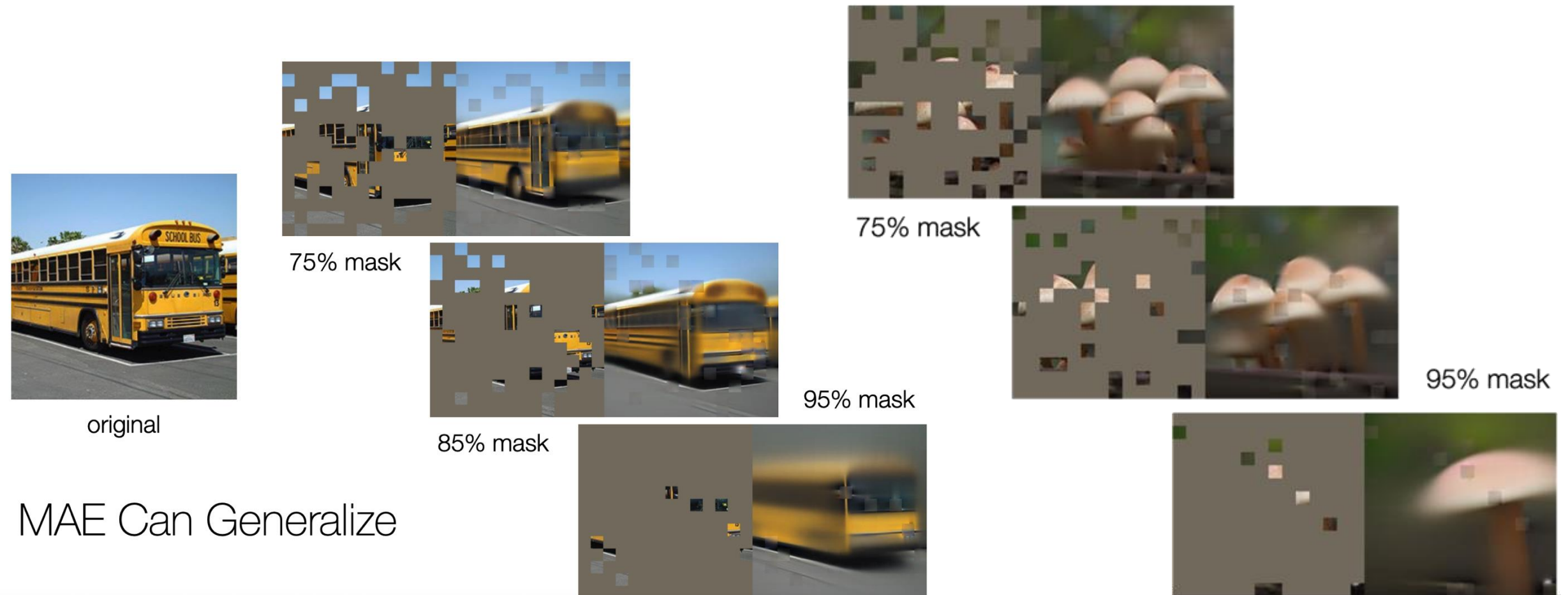
Pretext Task: Masked Generation

COCO val set (unseen)



MAE can
generalize!

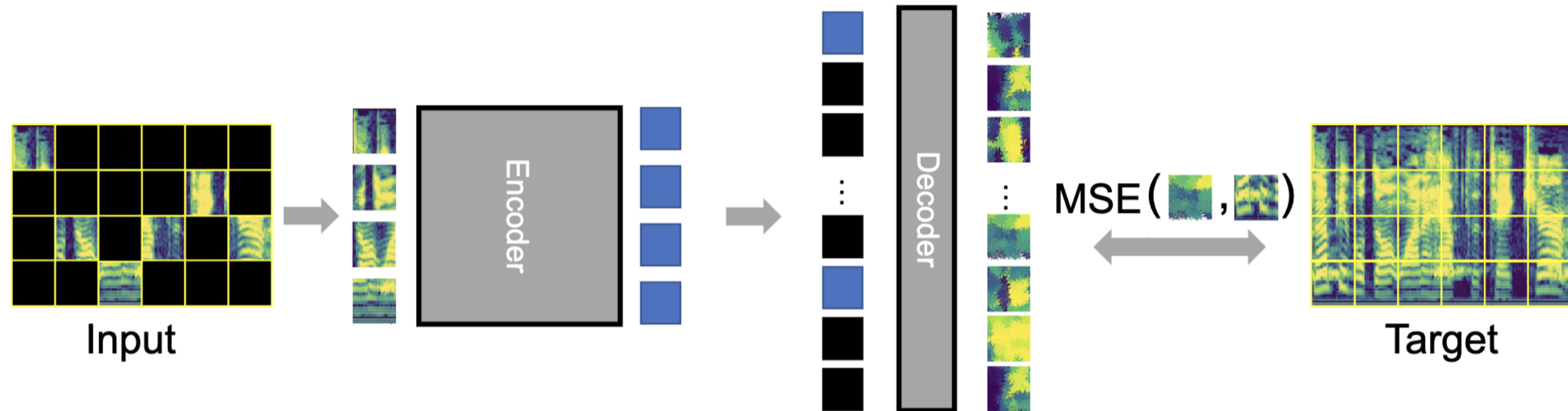
Pretext Task: Masked Generation



MAE Can Generalize

Very good representations for localization and counting!

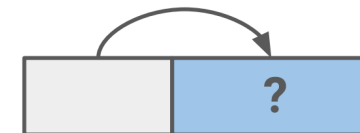
Pretext Task: MAE that listen



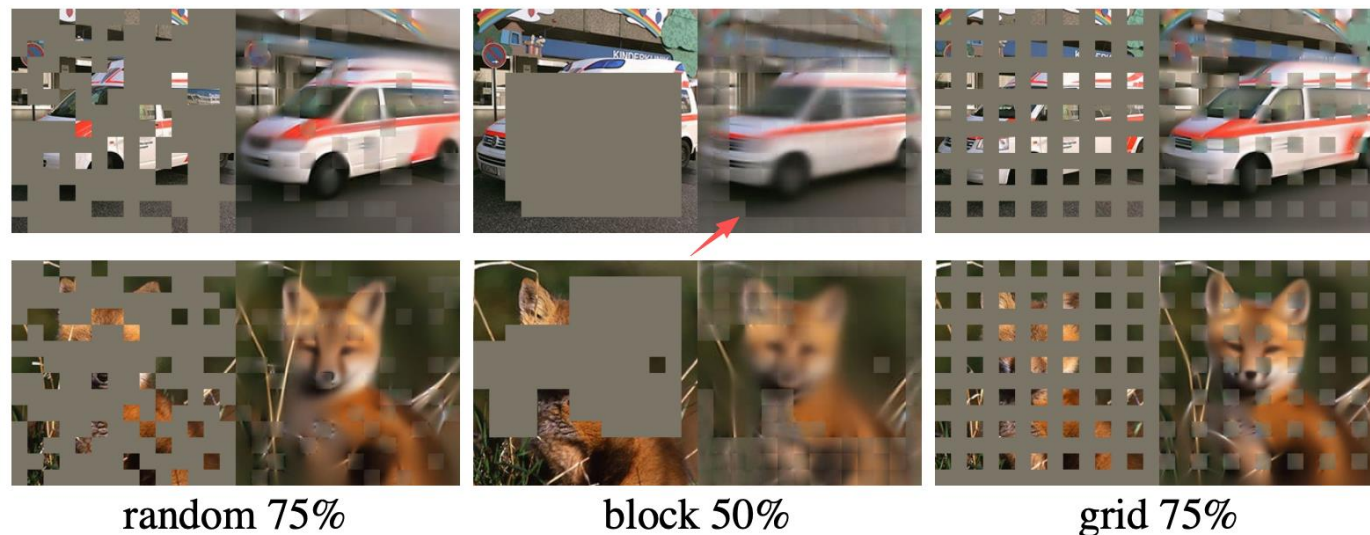
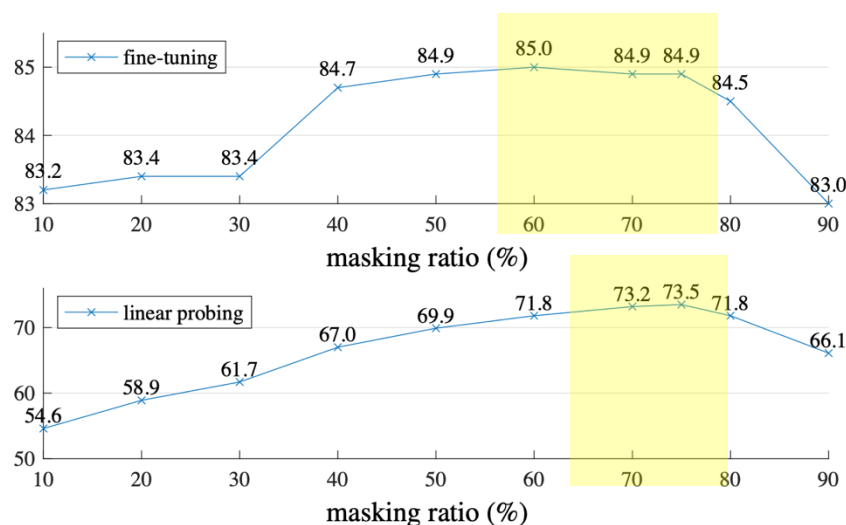
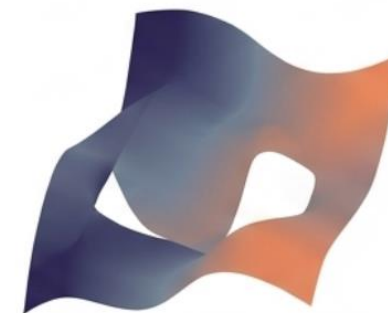
Masked Input

Why Masking helps?

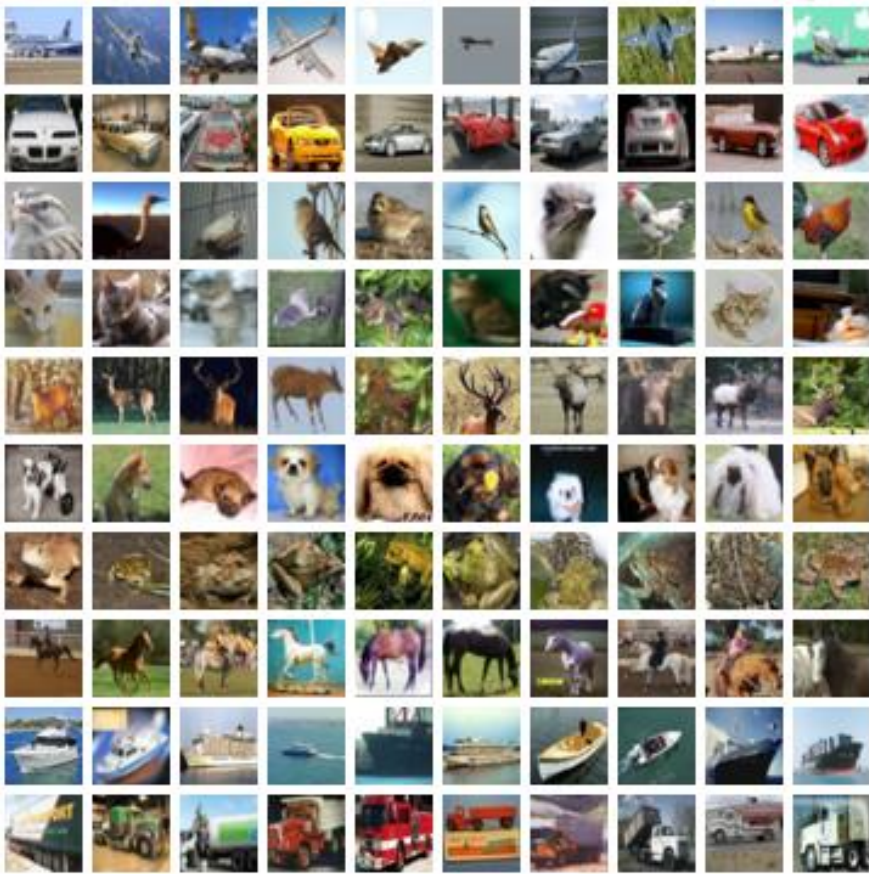
- Encourages **global** reasoning from limited context
- Prevents **trivial** reconstruction (shortcuts) by local pixel copying
- Low mask ratio -> learning to interpolate
- A masking ratio around **75%** provides the best trade-off
- **Random** masking better than block masking



"Intra-sample" prediction



Self-supervised Learning (SSL)



Encoder

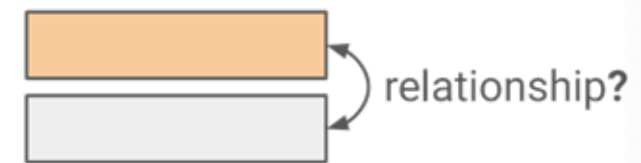
(1) Self-prediction



"Intra-sample" prediction

Pretext
Task

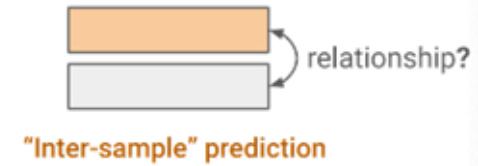
Note the "real" task (like classification) we care about



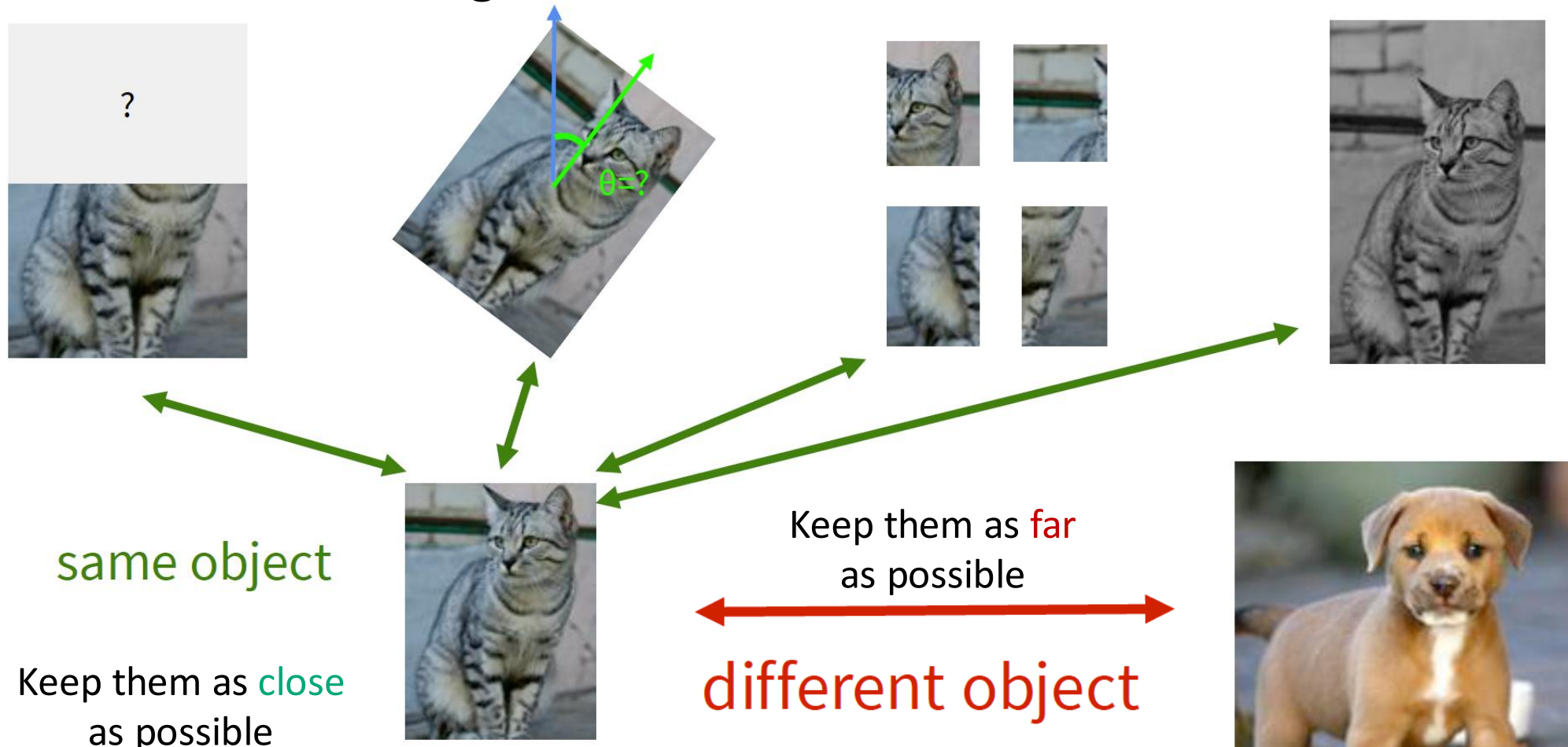
"Inter-sample" prediction

(2) Contrastive Learning

A more general pretext task?



Contrastive learning

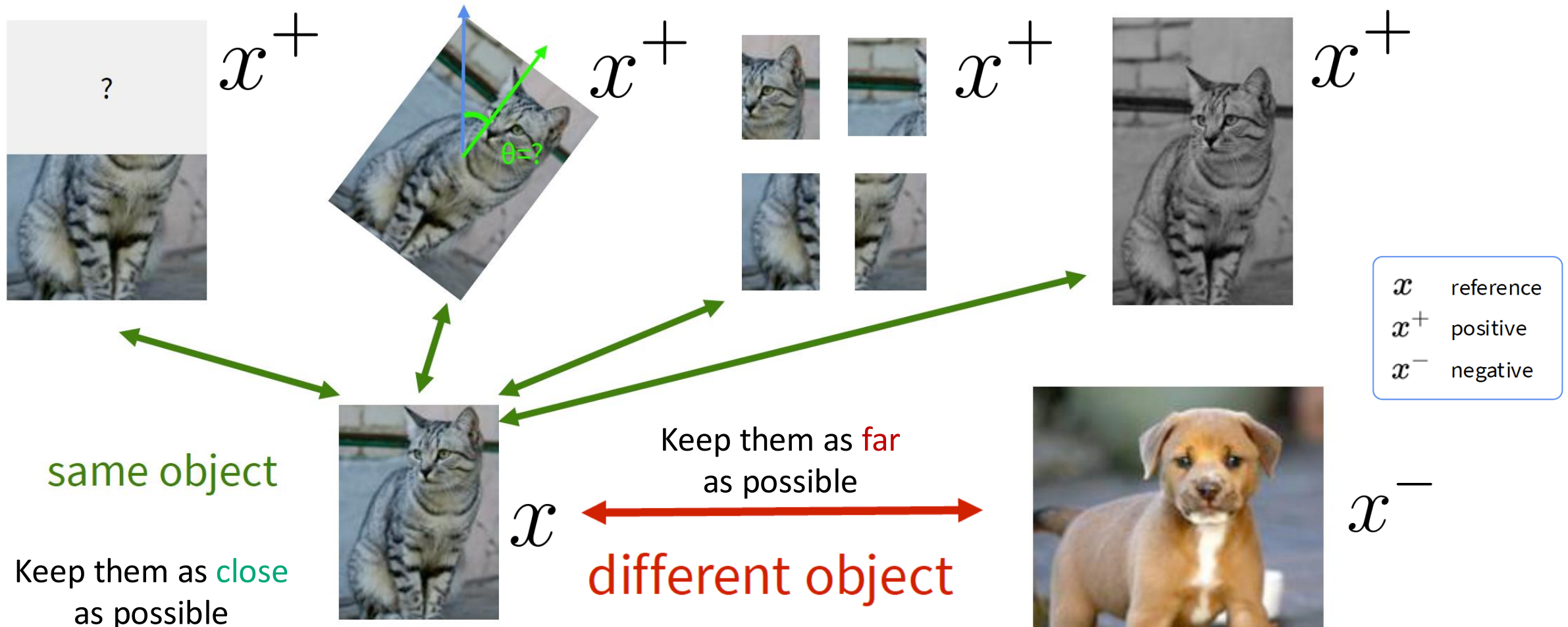


Contrastive Learning

$$d(f(x), f(x^+)) \ll d(f(x), f(x^-))$$

$f(\cdot)$ is the neural network encoder.

$d(\cdot)$ is a distance or similarity metric (often Cosine Similarity)

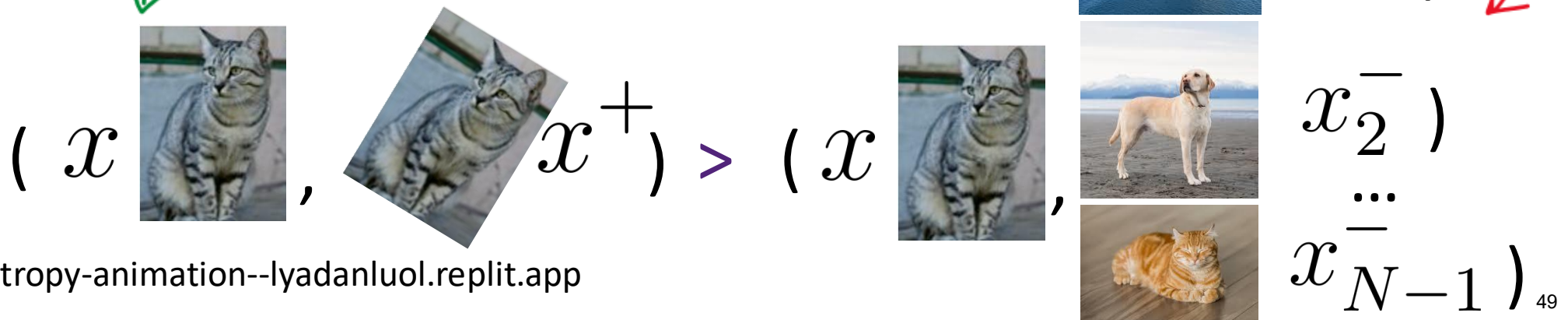


Contrastive Learning

Loss function given 1 positive sample and N-1 negative samples:

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+)))}{\exp(s(f(x), f(x^+))) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))} \right]$$

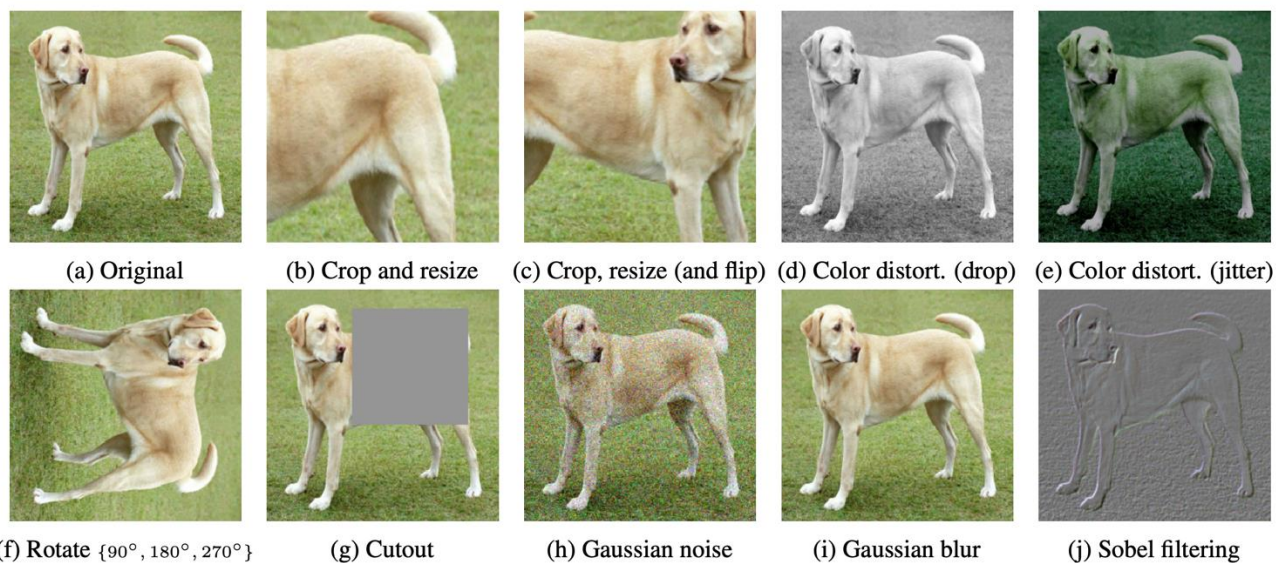
1. **Cross entropy loss:** for classification (correctly classify *positives* against *negatives*)
2. **Softmax:** normalize logits to distribution



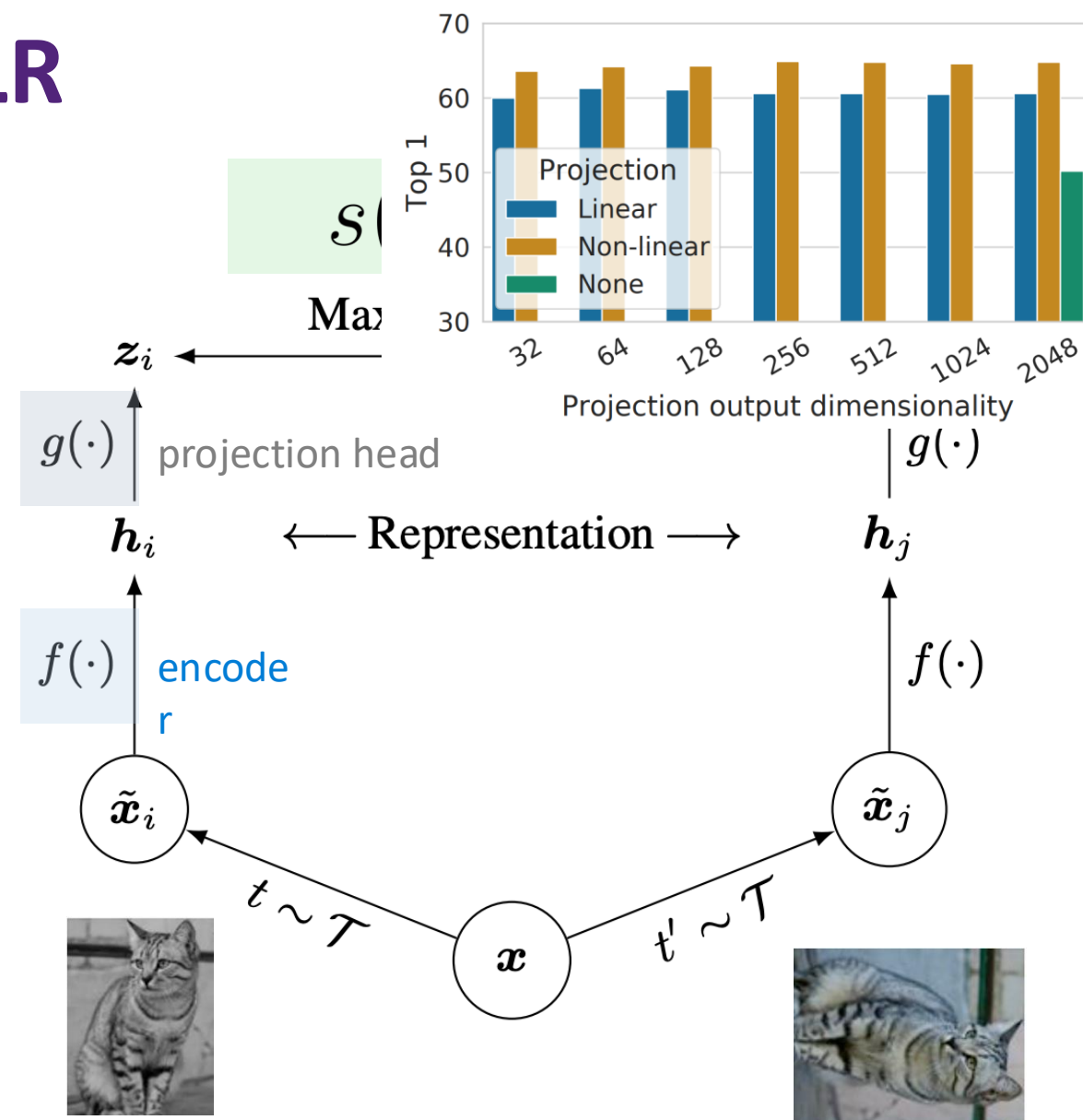
Contrastive Learning: SimCLR

Cosine similarity as the score function

$$s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$$



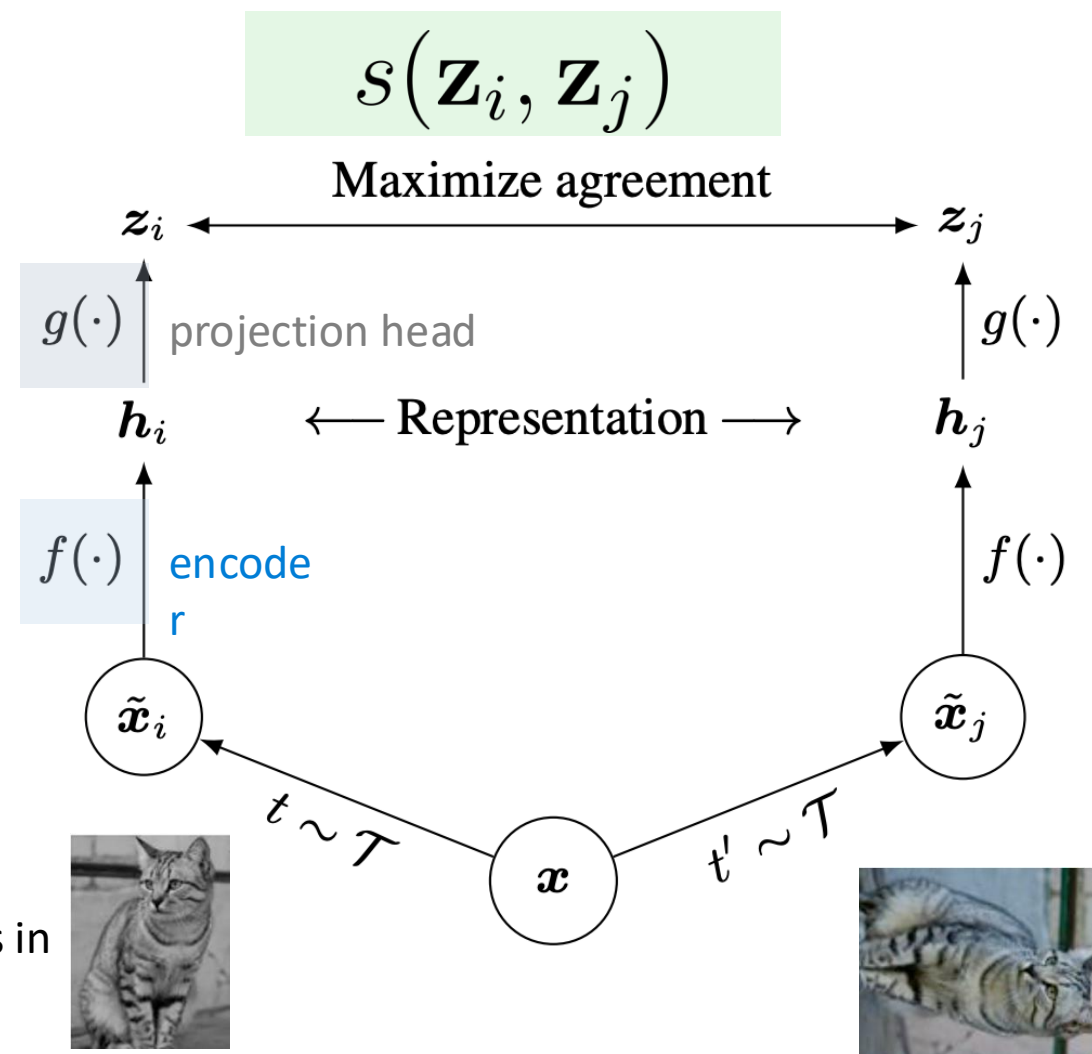
Generate positive samples through data augmentation



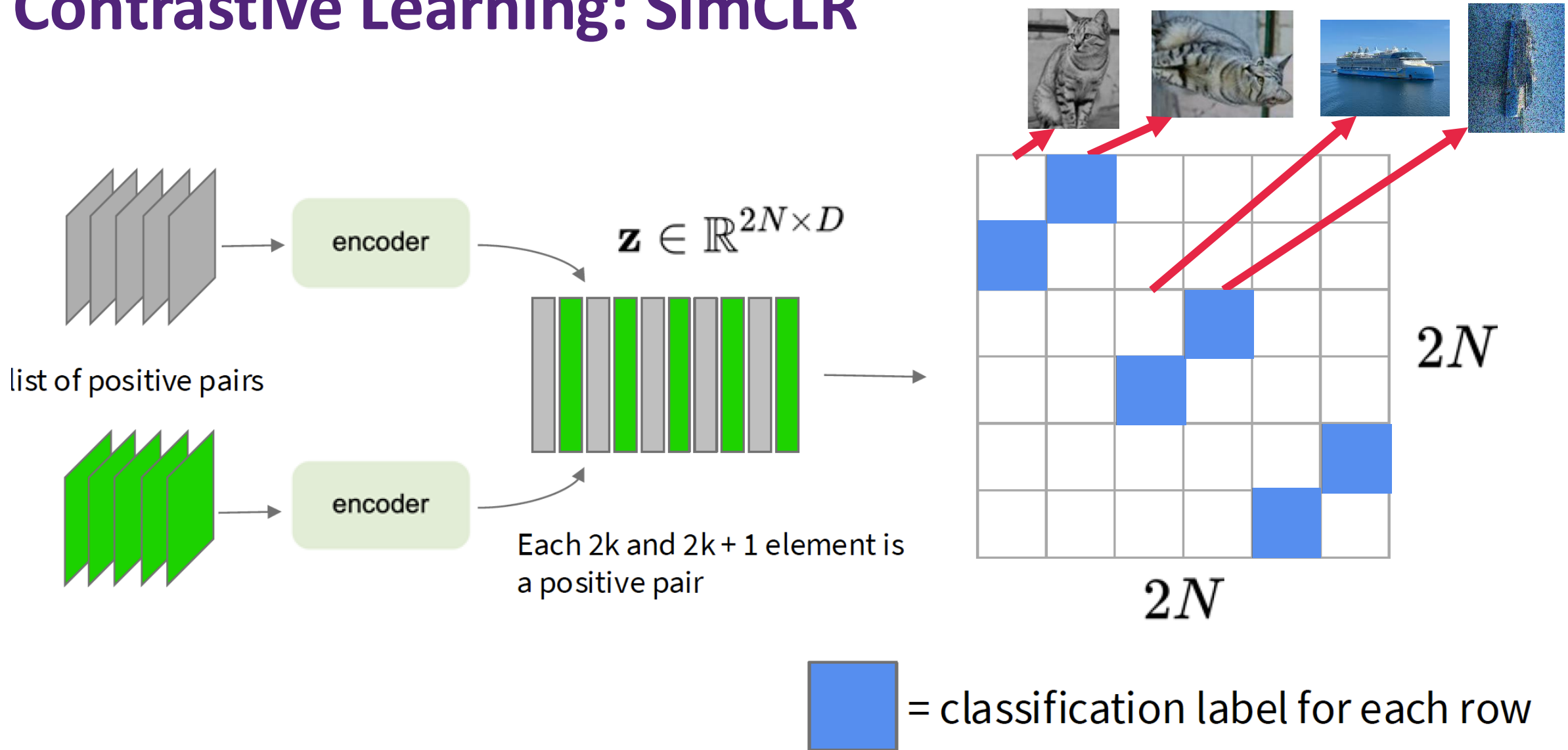
Contrastive Learning: SimCLR

Algorithm 1 SimCLR's main learning algorithm.

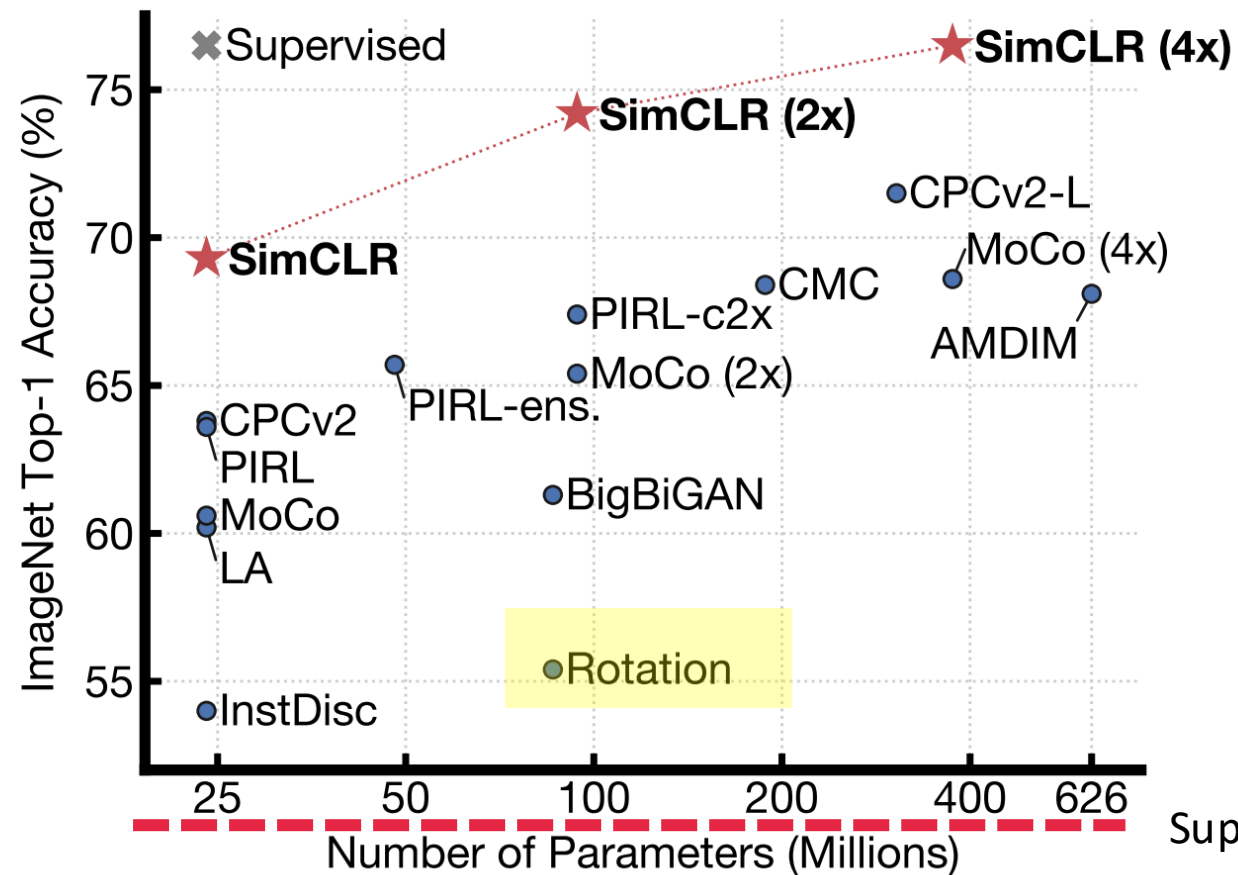
input: batch size N , constant τ , structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$
 $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$ # representation
 $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$ # projection
 # the second augmentation
 $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$
 $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$ # representation
 $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$ Any other samples in batch as *negative*
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$



Contrastive Learning: SimCLR



Contrastive Learning: SimCLR



Self-supervised Pretraining

Train feature encoder on ImageNet (*entire* training set) using SimCLR.

Downstream Finetuning

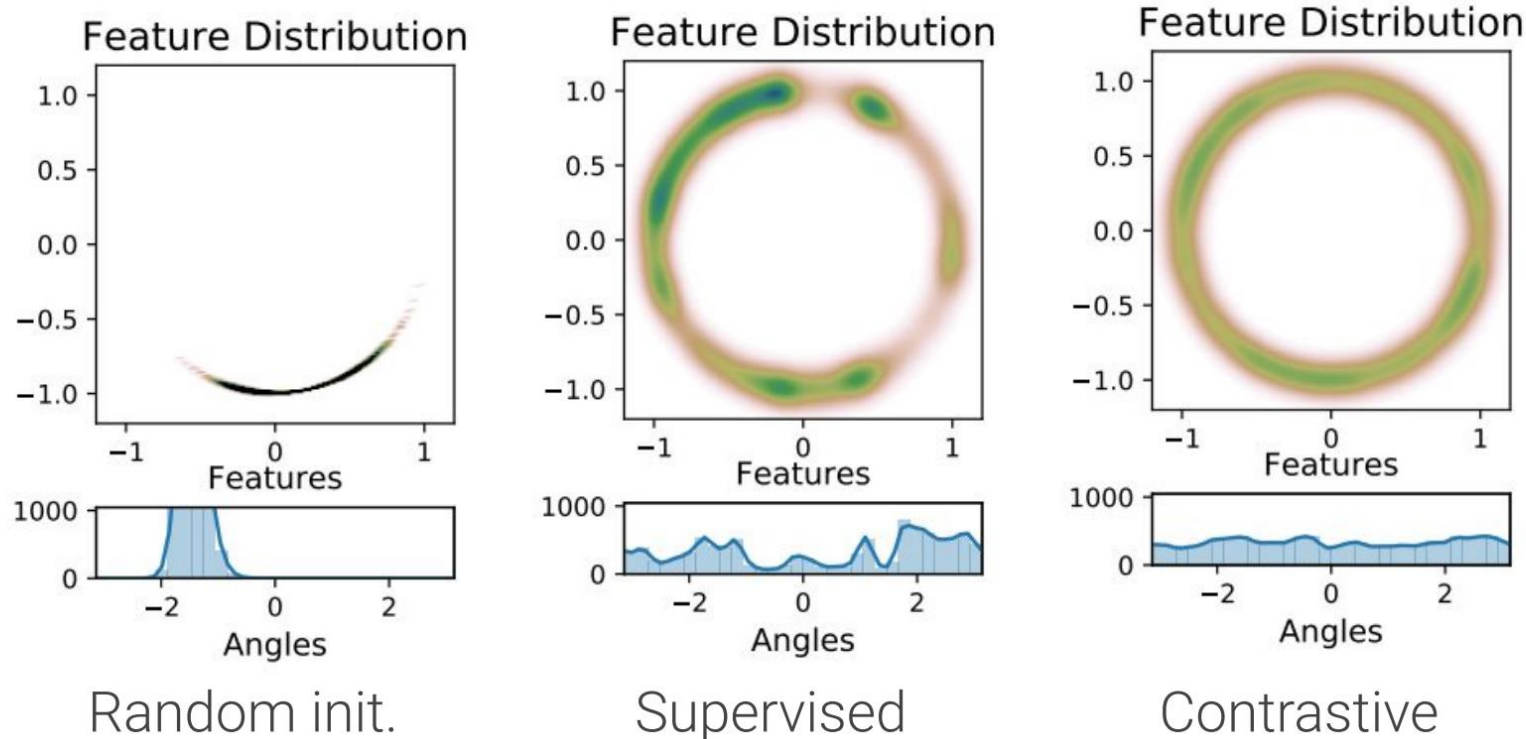
Freeze feature encoder, train a linear classifier on top with labeled data (1%).

Supervised with 1% Labeled data

Why Contrastive Learning is Good

Contrastively learned features are more **uniform** and **aligned**.

- **Uniform:** features should be distributed uniformly on the hypersphere S^d
- **Aligned:** features from two views of the same input should be the same

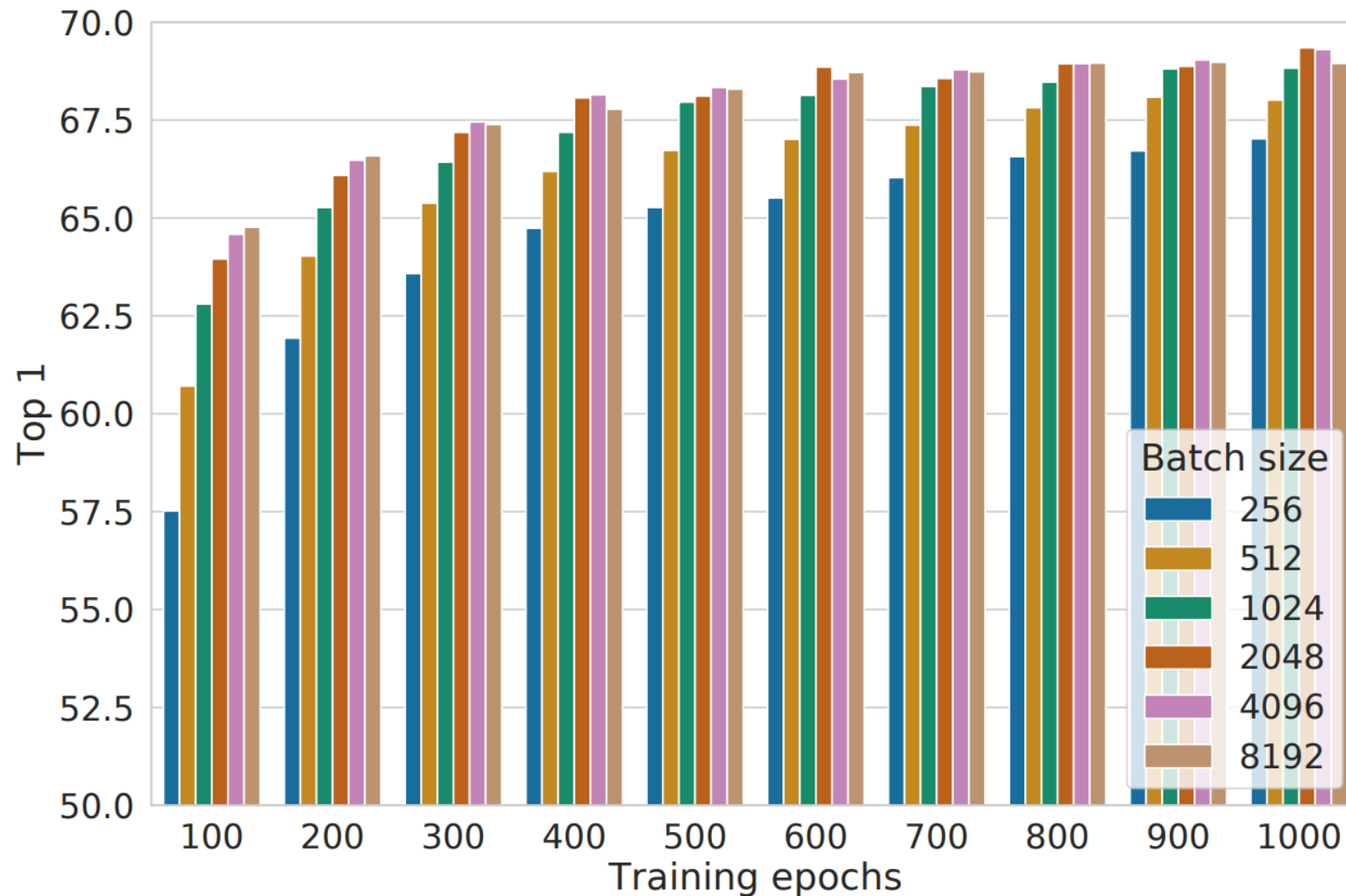


Softer classification
(*million of instances as classes*)
Better coverage
(*manifold*)

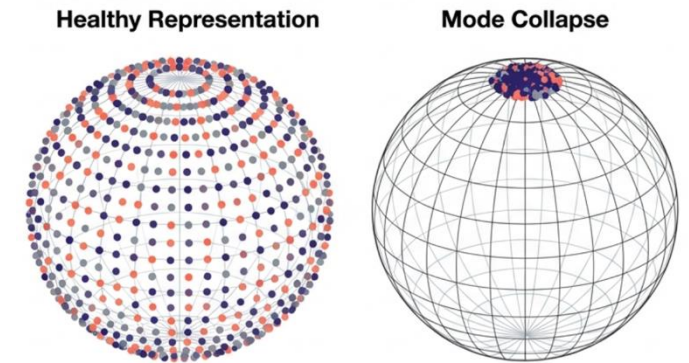


Large Batch Size

Contrastive Learning needs Large Batch Size

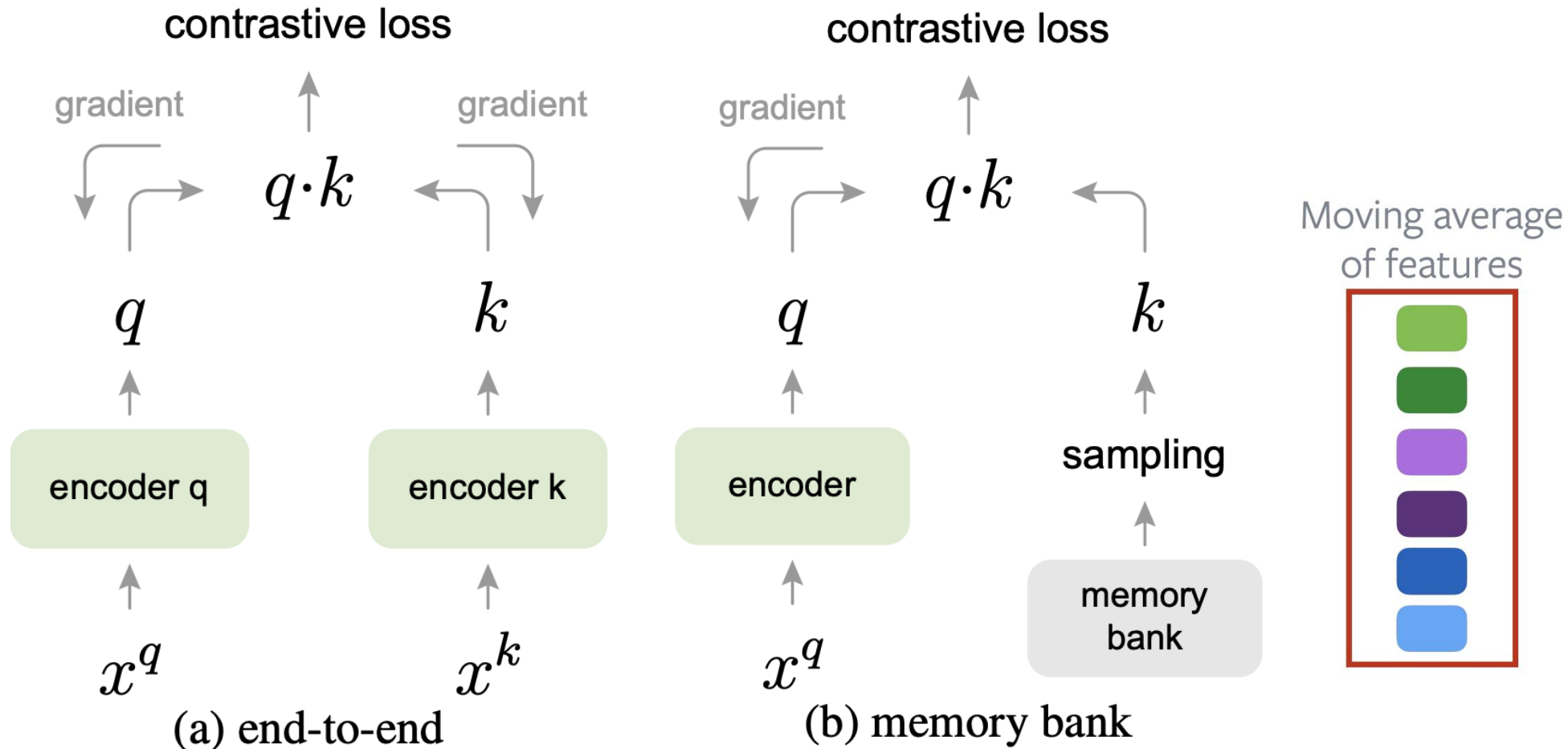


When you lack negative pairs...
models lazily projects everything
to local area (**shortcuts / trivial
solution**)



Harmful for downstream tasks

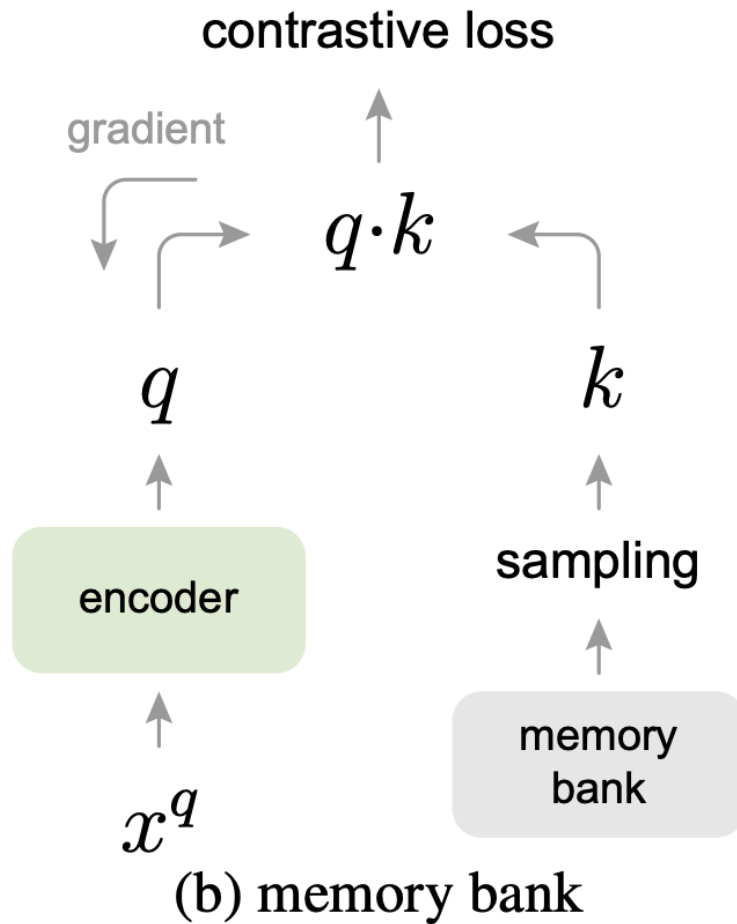
Momentum Contrast (MoCo)



Need Large Batch Size!

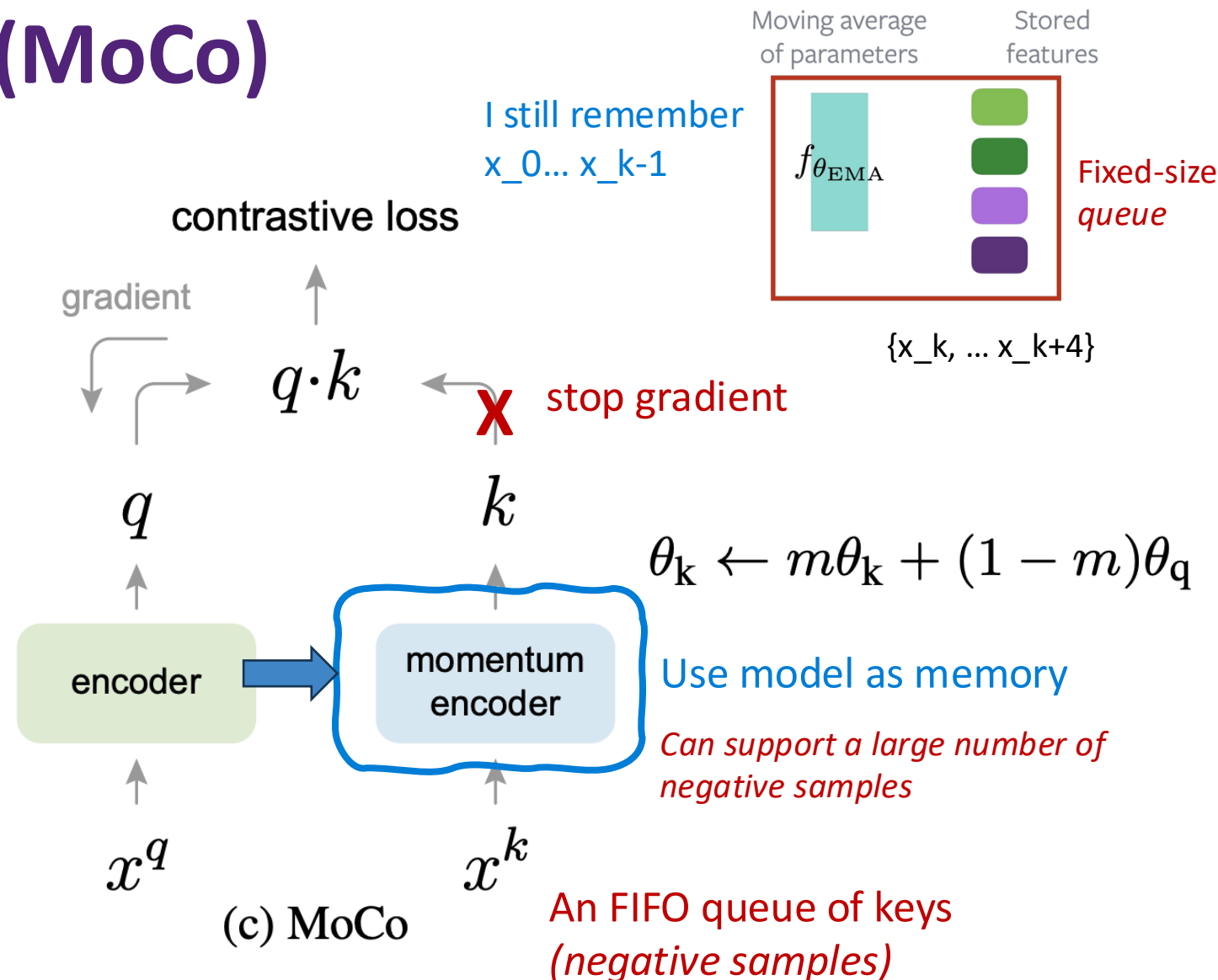
Simple :) Let's store them in memory

Momentum Contrast (MoCo)



Simple :) Let's store them in memory

😬 could be insane...



Self-Distillation with No Labels (DINO)

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron^{1,2} Hugo Touvron^{1,3} Ishan Misra¹ Hervé Jegou¹
Julien Mairal² Piotr Bojanowski¹ Armand Joulin¹

¹ Facebook AI Research ² Inria* ³ Sorbonne University

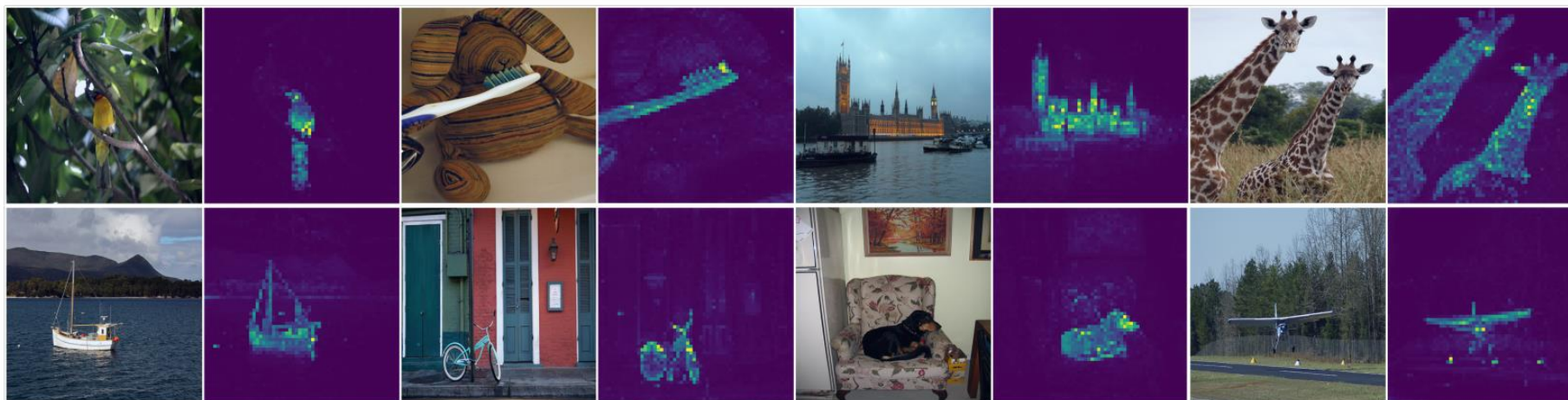
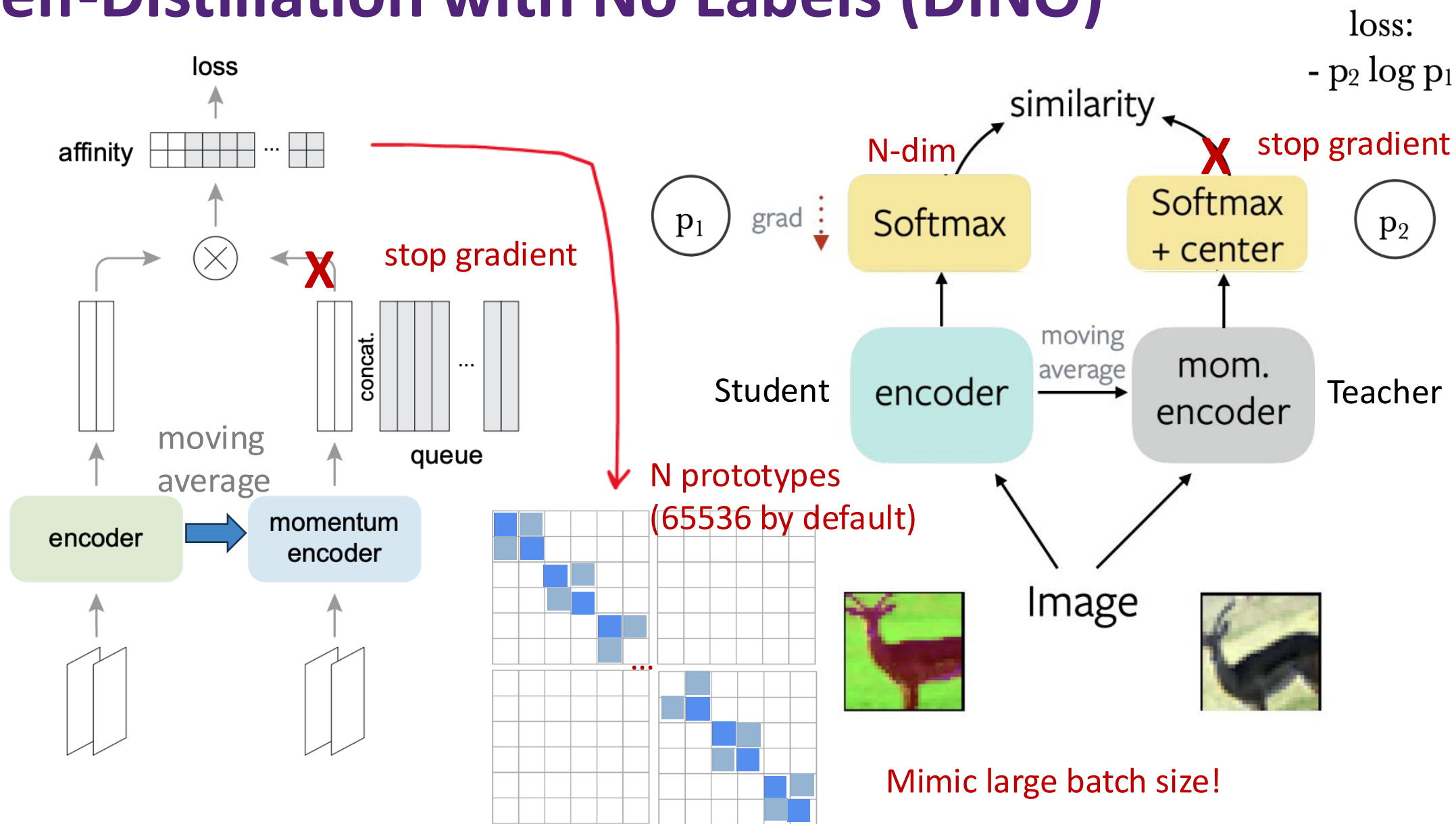


Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

Self-Distillation with No Labels (DINO)



Self-Distillation with No Labels (DINO)

How to prevent feature collapse?

1. Centering

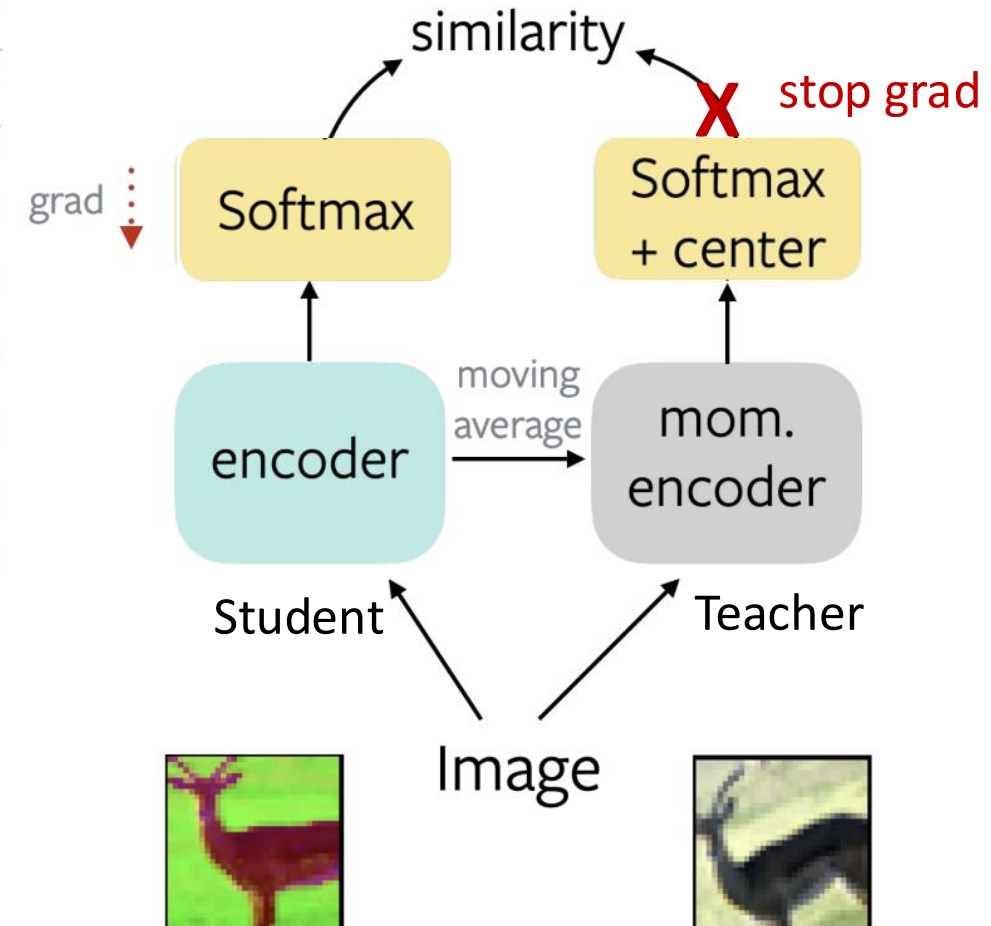
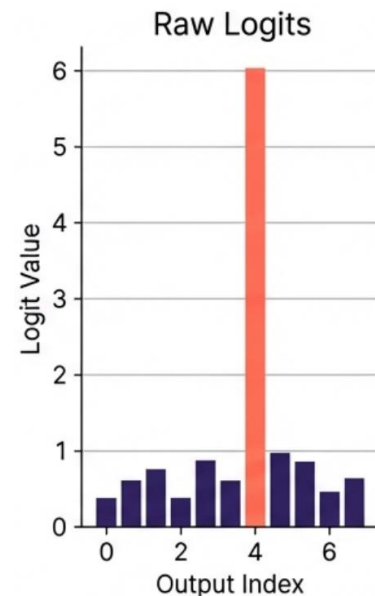
Teacher's outputs are centered by a running mean c

Teacher's output

$$p_t = \text{softmax} \left(\frac{z_t - c}{\tau_t} \right)$$

Prevents one prototype (since we don't have label) dominating the softmax

$$c \leftarrow mc + (1 - m) \text{mean}(z_t)$$



Self-Distillation with No Labels (DINO)

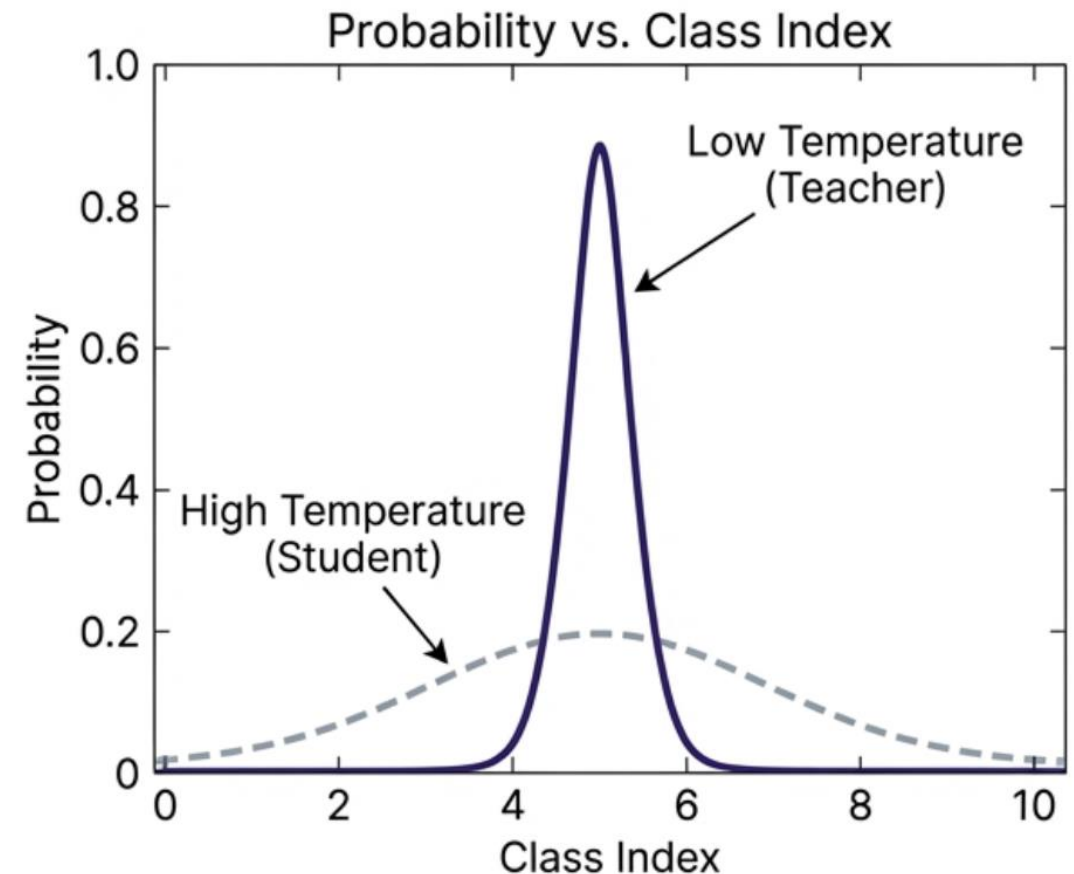
How to prevent feature collapse?

2. Sharpening

Centering alone leads to uniform (lazy) distributions

- Force student to make confident and peaky prediction
- Balance: centering (diversity) vs. sharpening (confidence)

$$p = \text{softmax}\left(\frac{g}{\tau_{\text{teacher}}}\right) \text{ where } \tau_{\text{teacher}} < \tau_{\text{student}}$$

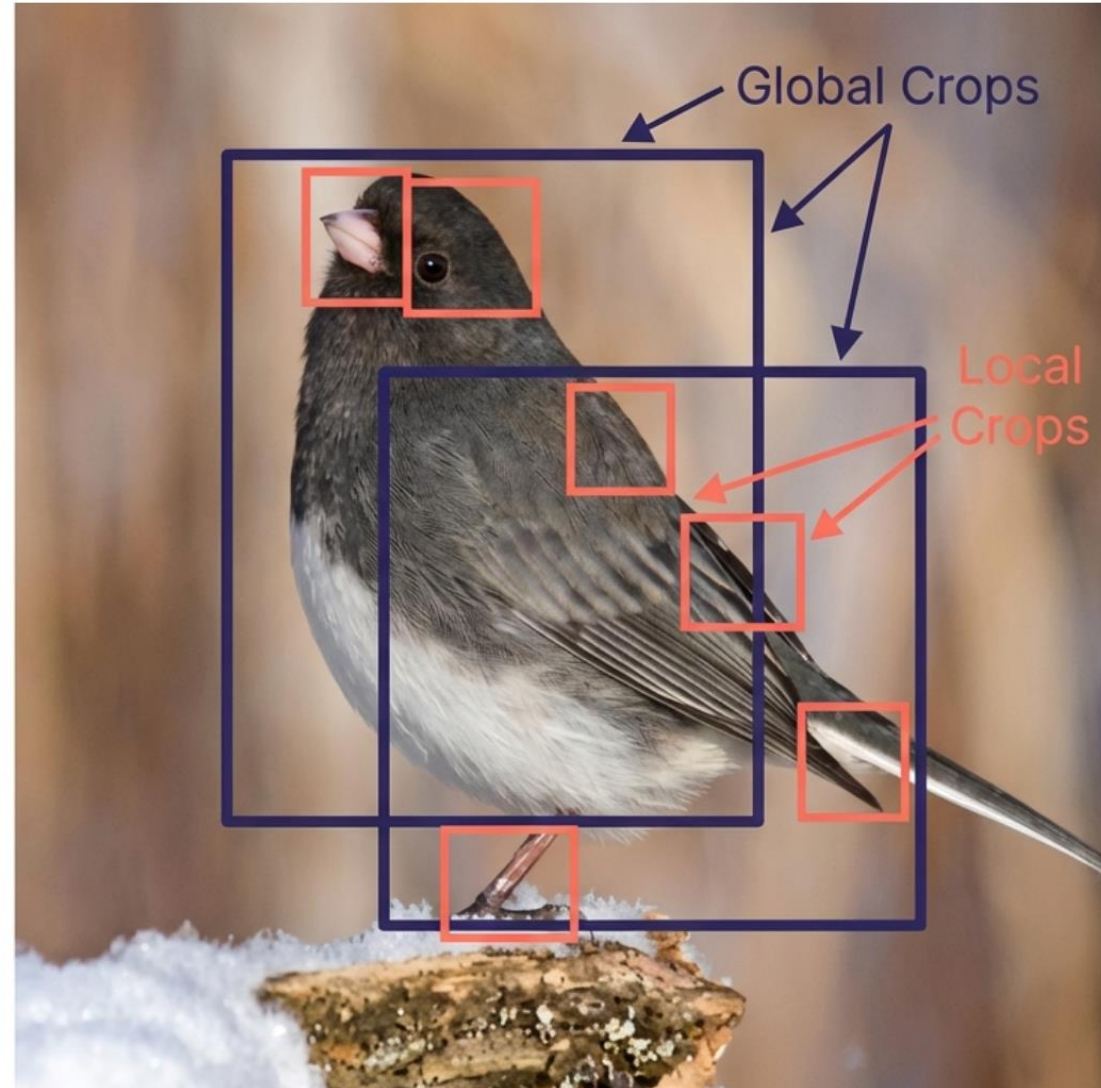


Self-Distillation with No Labels (DINO)

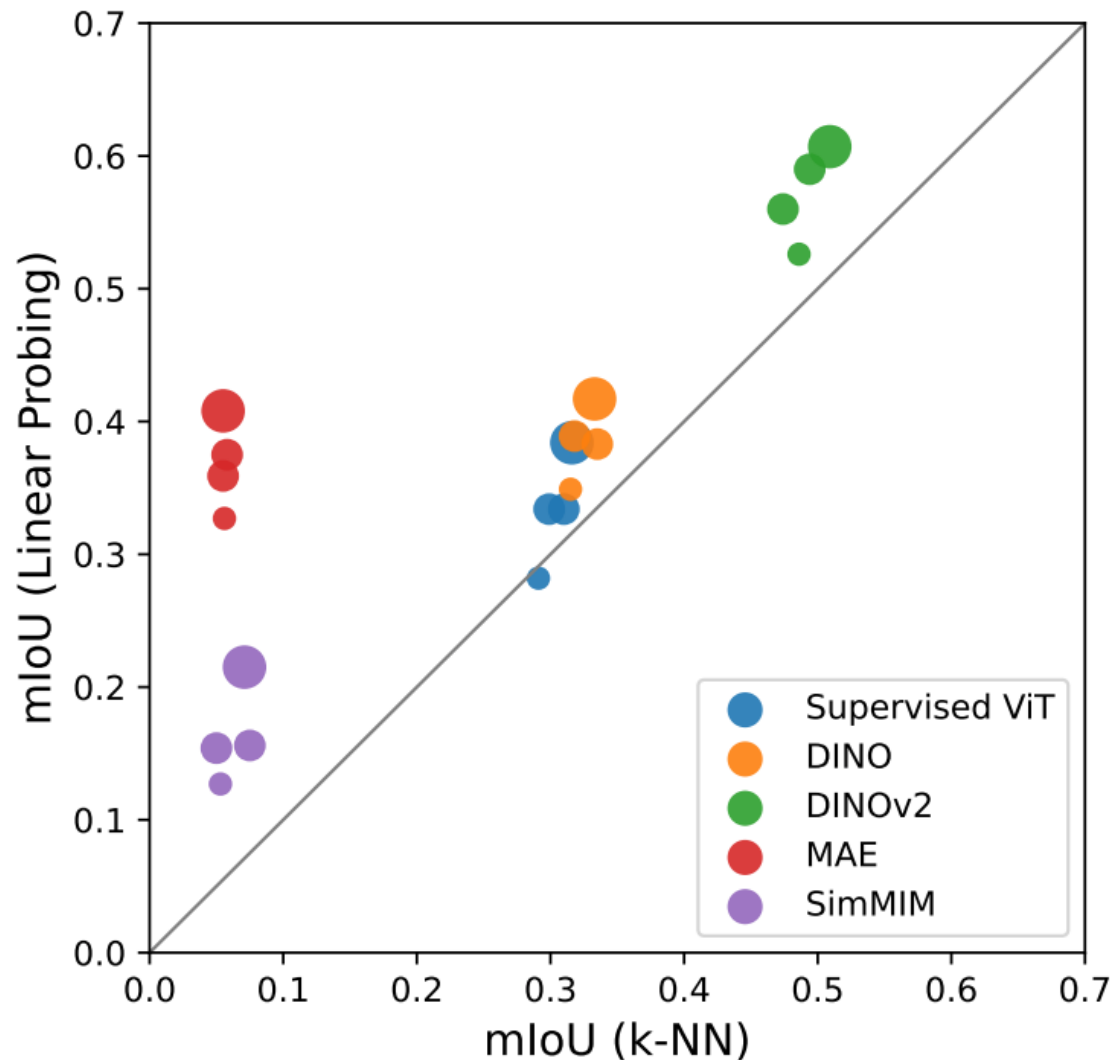
Multi-crop Strategy: Local-to-Global

- Global views (x2): Cover >50% of the image.
- Local views (x8): Cover small regions (<50%).
- Student sees all; Teacher only sees Global.
- Enforces: "This small part belongs to this object."

$$\mathcal{L} = \sum_{g \in \mathcal{G}} \sum_{l \in \mathcal{L}} \text{Loss}(\text{Student}(l), \text{Teacher}(g))$$

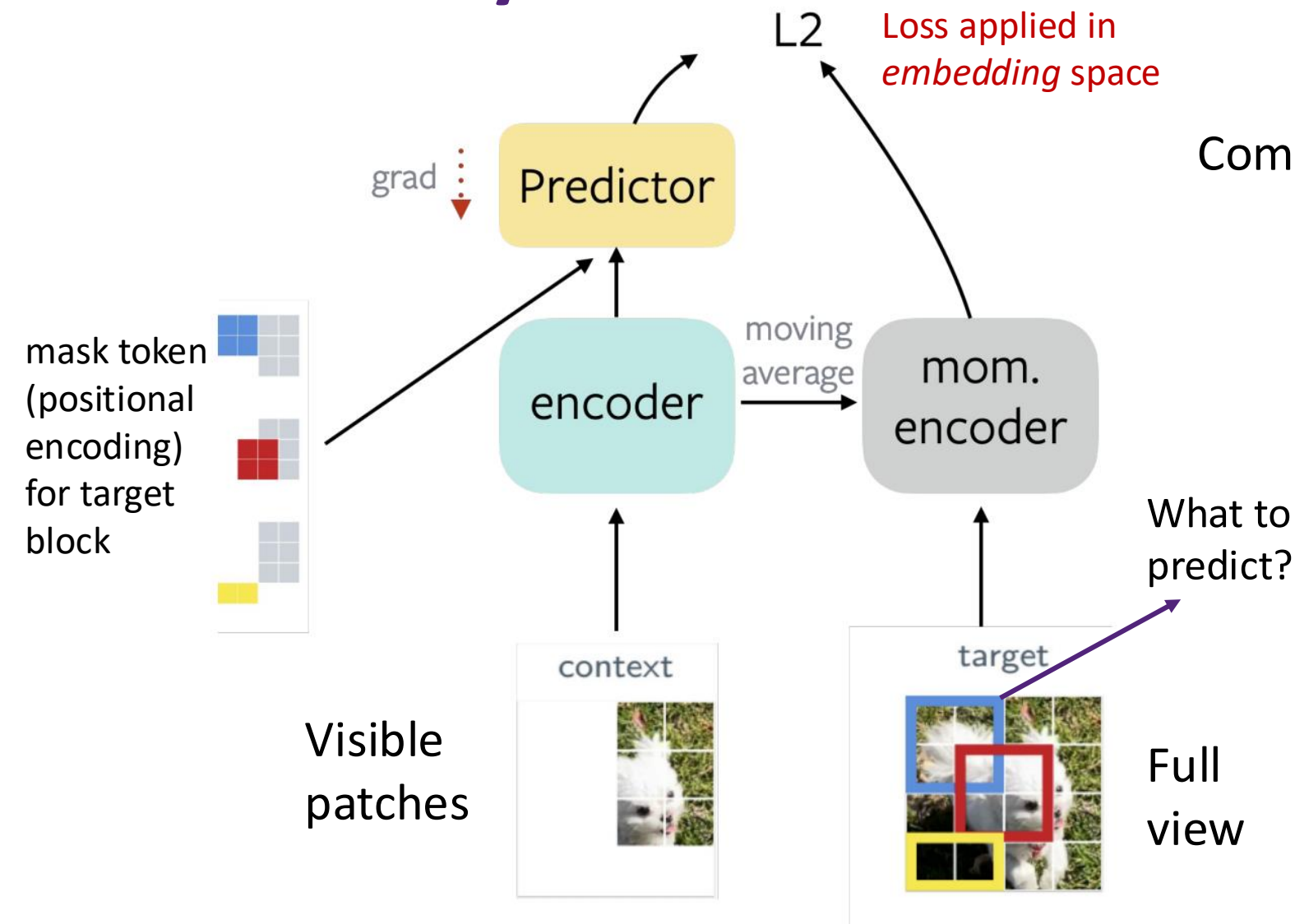


DINO vs MAE

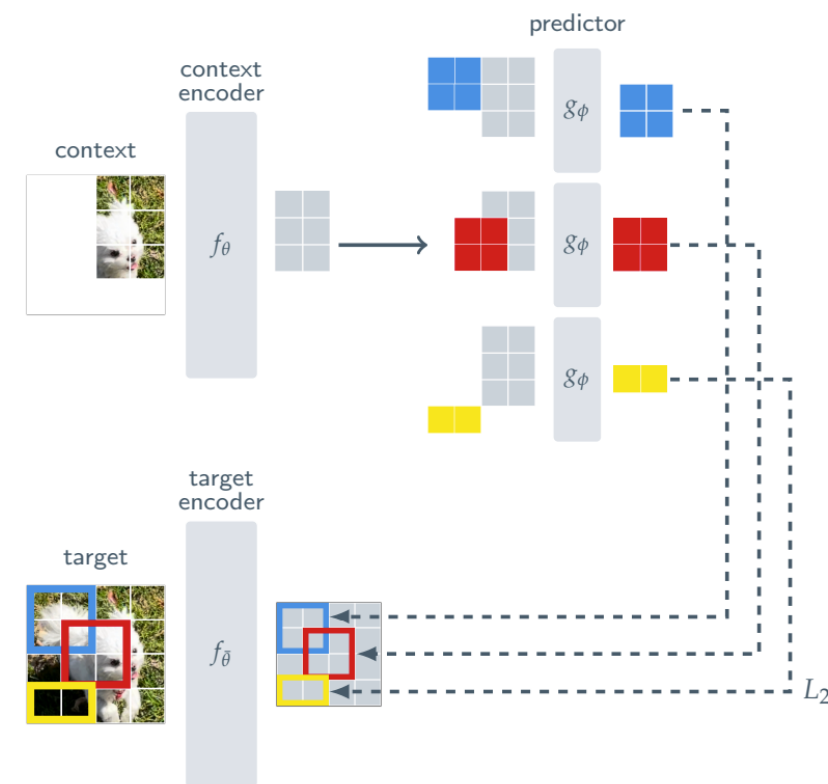


- DINO produces **more universal** patch representations
- MAE embeddings have **higher variance** components
- That hurts distance-based tasks (k-NN, clustering)
- DINO features work better for downstream tasks *without finetuning*
- **Modern segmentation foundation models** (Segment Anything SAM) use DINO features

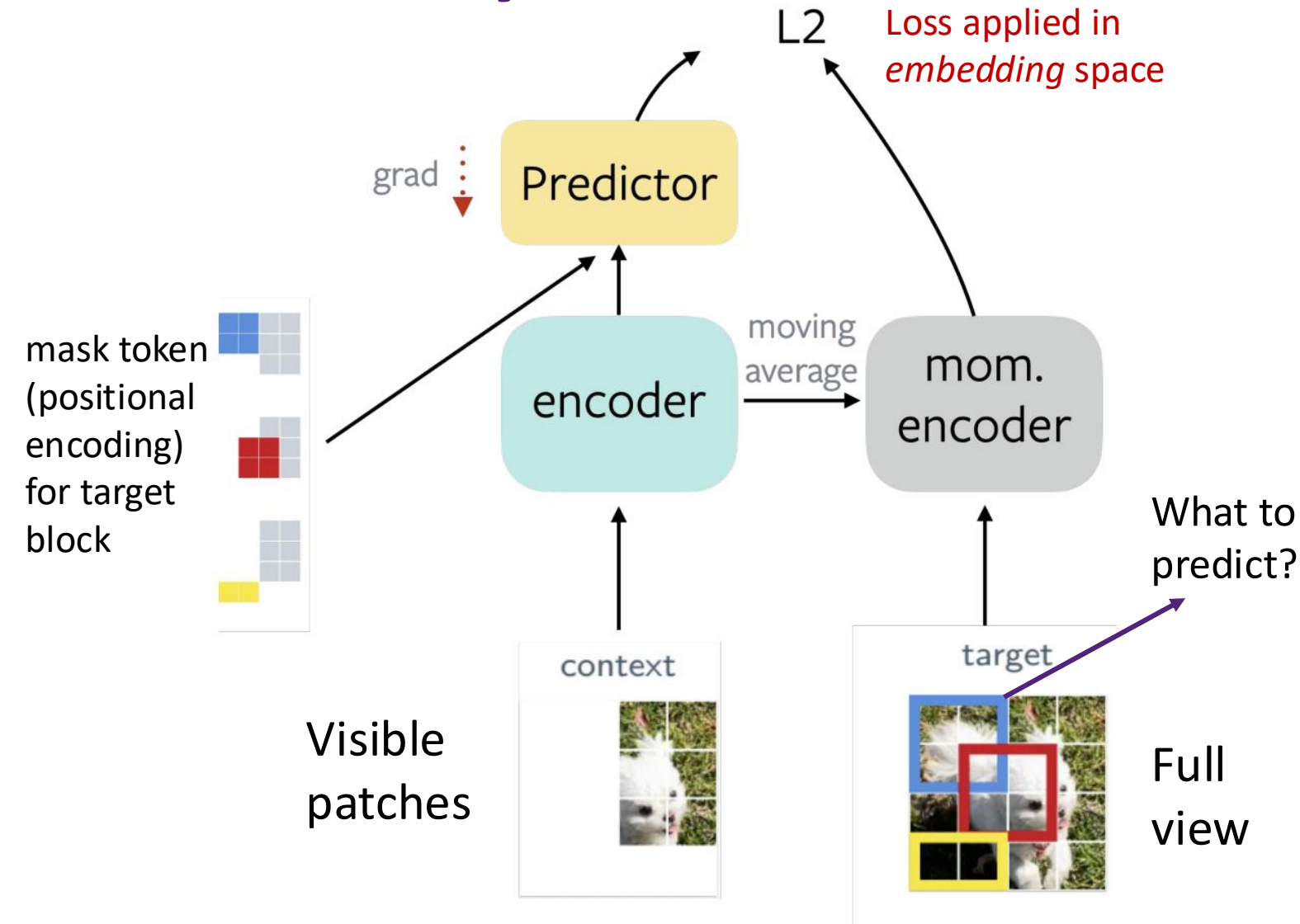
What's beyond... I-JEPA



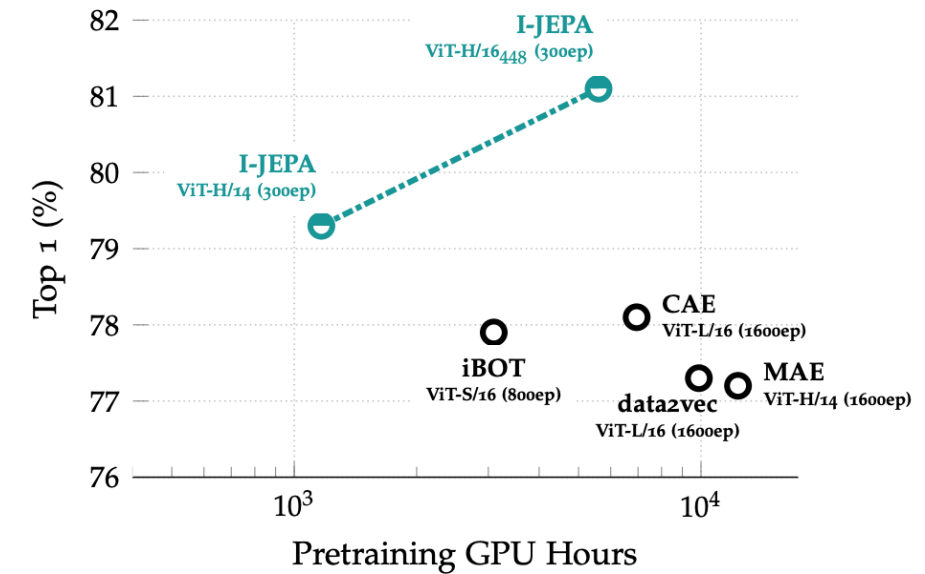
Combine the idea of DINO and MAE



What's beyond... I-JEPA



ImageNet-1K Linear Evaluation vs GPU Hours



Method	Arch.	CIFAR100	Places205	iNat18
<i>Methods without view data augmentations</i>				
data2vec [8]	ViT-L/16	81.6	54.6	28.1
MAE [36]	ViT-H/14	77.3	55.0	32.9
I-JEPA	ViT-H/14	87.5	58.4	47.6
<i>Methods using extra view data augmentations</i>				
DINO [18]	ViT-B/8	84.9	57.9	55.9
iBOT [79]	ViT-L/16	88.3	60.4	57.3

What we covered in this week...

After today's class, you should be able to

- explain why **scaling in vision** is more challenging than in language
- understand the motivation and main families of **self-supervised learning**
- compare different SSL paradigms, including
 - ❑ **Pretext tasks** (self-prediction: rotation, position, colorization)
 - ❑ **Reconstruction-based** (MAE)
 - ❑ **contrastive-style** (SimCLR) and **teacher-student** (MoCO, DINO) methods
- recognize how recent methods such as **I-JEPA** move beyond pixel reconstruction toward prediction in embedding space

Next Week: Multimodal Alignment...



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

THANKS!

Do you have any questions?

uqyluo@uq.edu.au

The University of Queensland

